COURS BASE DE DONNÉES RELATIONNELLES, LANGAGE SQL ET STRUCTURE DE DONNÉES II

Table des matières

CHAPITRE 1 : Rappels et Fondements des Bases de Données 4
I. Modèle conceptuel (MCD) et modèle logique (MLD) 4
1) Le Modèle conceptuel de données (MCD) 4
2) Le modèle logique de données (MLD) 7
II. Normalisation (1NF, 2NF, 3NF, BCNF) 10
1) Présentation 11
2) La première forme normale (1NF) 12
3) La deuxième forme normale (2NF) 12
4) La troisième forme normale (3NF) 12
5) La forme normale de Boyce Codd (BCNF) 12
III. Concepts de tables, clés, relations, intégrité référentielle 12
1) Concepts de tables 12
2) Relations 13
3) Clés 13
4) Intégrité référentielle 13
CHAPITRE 2 : Langage SQL Avancé 15
I. Présentation du SQL 15
II. Requêtes conditionnelles et expressions CASE 15
1) Requêtes conditionnelles 15
2) Expressions CASE 16
III. Requêtes complexes et les vues 18
1) Les jointures 18
2) Les Sous-requêtes 22
3) Les vues 24
IV. Fonctions d'agrégation et fonctions analytiques (OVER, PARTITION) 24
1) Fonctions d'agrégation 24
2) Fonctions analytiques 26
V. Gestion des dates, chaînes, conversions et expressions régulières 32
1) Gestion des dates 32
2) Chaînes 33
3) Conversions 34
4) Expressions régulières 34
CHAPITRE 3 : Programmation SQL 36
I. les notions de base 36
A. Structure de PL/pgSQL 36
B. Affectation et affichage 37
C. Déclaration de variables 38
D. Structures de contrôle 41
II. Création de procédures et fonctions stockées 47
A. Les fonctions 47
B. Les procédures 50
III. Création et utilisation des triggers 52
IV. Transactions & contrôle de concurrence (COMMIT, ROLLBACK, SAVEPOINT) 52
V. Gestion des droits et sécurité des accès (GRANT, REVOKE) 54
CHAPITRE 4 : Optimisation de Performance 56
I. Indexation (B-Tree, Hash, Index composés) 56
1) Présentation 56

2) Création des index 56
3) Les différents types d'index 57
II. Plans d'exécution (EXPLAIN, ANALYZE) 58
1) Présentation 58
2) Composants d'un plan d'exécution de requête 58
3) Comment générer un plan d'exécution de requête 58
4) Interprétation d'un plan d'exécution de requête 58
5) Problèmes courants identifiés dans les plans d'exécution de requêtes 59
6) Optimisation des requêtes à l'aide de plans d'exécution 59
7) SQL EXPLAIN 59
8) EXPLAIN ANALYZE 61
III. Optimisation des requêtes SQL 61
IV. Partitionnement et organisation physique 62
A. Partitionnement 62
1) Partitionnement vertical 62
2) Partitionnement horizontal 62
3) Outils de partitionnement des données 63
B. Organisation physique 64
1) Notion de chemins d'accès 64
2) Méthodes d'accès standards du système d'exploitation 65
3) Méthodes d'accès non standards 65
CHAPITRE 5 : Structures de Données 66
I. Généralité 66
II. Rappel en langage C 66
A. Les tableaux 66
1) Définition 66
2) Tableaux statiques 66
3) Tableaux dynamiques 66
B. Les Structures 67
C. Types synonymes 68
D. Pointeurs 68
1) Présentation 68
2) Pointeurs et tableaux 69
3) Allocation dynamique 69
III. Listes 70
E. Listes chaînées 70
1) Présentation 70
2) Procédures de manipulation de listes 71
3) Listes doublement chaînées 73
4) Listes circulaires 73
F. Piles 74
G. Files 74
IV. Arbres binaires, AVL, B-Tree 75
A. Généralités 75
B. Arbres binaires 76
1) Présentation 76
2) Algorithmes simples 77
a) Taille de l'arbre 77
b) Hauteur de l'arbre 78
c) Parcours en profondeur 78

d) Parcours en largeur 78
C. Arbre AVL 78
D. B-Tree 79
1) Définition : 79
2) Recherche dans un B-arbre : 81
3) Insertion dans un B-arbre : 81
4) Suppression dans un B-arbre 82
V. Tables de hachage 82
VI. Choix des structures en fonction des usages (complexité) 83
1) Présentation 83
2) Facteurs de choix 84
CHAPITRE 6 : Intégration des Bases de Données dans les Applications 85
I. Connexion via ORM (ex. SQLAlchemy, Hibernate ou PDO) 85
A. Persistance objet avec JPA 85
1) Les entités 86
2) Un premier exemple d'utilisation de JPA 86
B. PDO : Interface d'accès aux BDD 93
1) Présentation 93
2) Avantages de PDO 93
3) Activation - Php Windows 93
4) Connexion : création et gestion d'erreur 94
5) Méthodes : exec et query 95
6) Méthode : prepare 97
7) Sécuriser sans passer par des requêtes préparées 97
II. Accès concurrents et verrouillages 98
III. Bonnes pratiques d'architecture applicative (MVC / 3-Tiers) 98
A. Le Modèle Vue Controleur 98
B. Client/Serveur 3-tiers 101
C. Architecture logique 102

CHAPITRE 1 : Rappels et Fondements des Bases de Données

I. Modèle conceptuel (MCD) et modèle logique (MLD)

1) Le Modèle conceptuel de données (MCD)

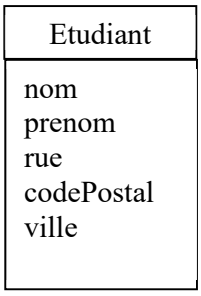
Le Système d'Information est le domaine dans lequel s'applique MERISE. Il est composé de moyens humains et techniques nécessaire au stockage et au traitement de l'information d'une organisation.

Le MCD est l'élément le plus connu de Merise et certainement le plus utile. Il décrit la signification des données sur lesquelles reposent le SI et les structures. Il permet d'établir une représentation claire de ces données. Il définit également les dépendances fonctionnelles de ces données entre elles.

a) Les entités

Une entité est une chose du monde réel qui possède une existence propre, concrète ou abstraite, des caractéristiques bien définis, qui présente un intérêt et a un sens pour le système d'information étudié. Il s'agit d'un objet matériel ou immatériel dans le SI .Exemple client, produit, étudiant..

Représentation graphique :



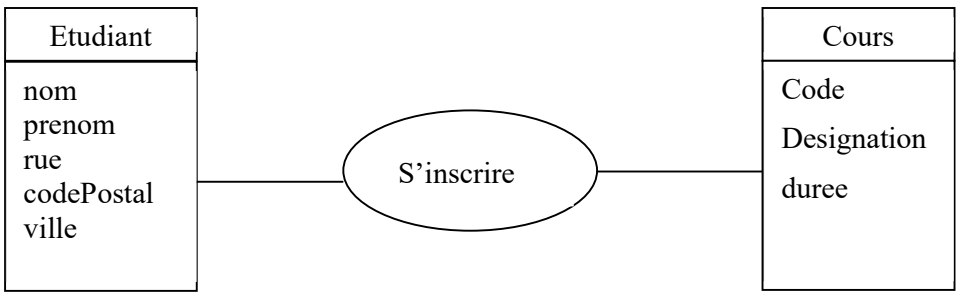
Pour faire le MCD, on devra notamment structurer les informations en les regroupant pour former les entités.

b) Les propriétés

Une propriété est une donnée élémentaire et pertinente pour le SI étudié. Le regroupement cohérent de propriétés définit les entités. Une entité est donc décrite par ses propriétés. C'est un attribut d'une entité, d'une relation ou association ; l'ensemble des propriétés du MCD compose **le Dictionnaire de Données.**

c) Les associations

Une association représente un lien sémantique entre entités. C'est une relation entre entités du SI, lien entre 2 ou plusieurs entités. On parlera d'association binaire si elle relie deux entités, de ternaire si elle relie trois entités. Elle n'a pas d'ordre de lecture spécifique, c'est pourquoi on le représentera par des verbes à l'infinitif.

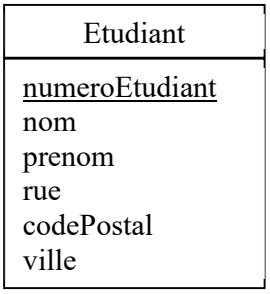


d) Les occurrences d'entités ou d'association

On appelle **Occurrence d'une entité**, un exemplaire, une réalisation (concrétisation) de cette entité. Par exemple le client nommé DUPONT est une occurrence de l'entité « client ». L'entité « client » aura donc autant d'occurrence que l'entreprise aura de clients.
Une occurrence d'entité aura une valeur pour chaque propriété de l'entité ; l'ensemble de ces valeurs caractérisent l'occurrence.
Une occurrence d'association est une réalisation de cette association. Donner une occurrence d'association reviendrait à donner une occurrence de chaque entité reliée.

e) L'identifiant

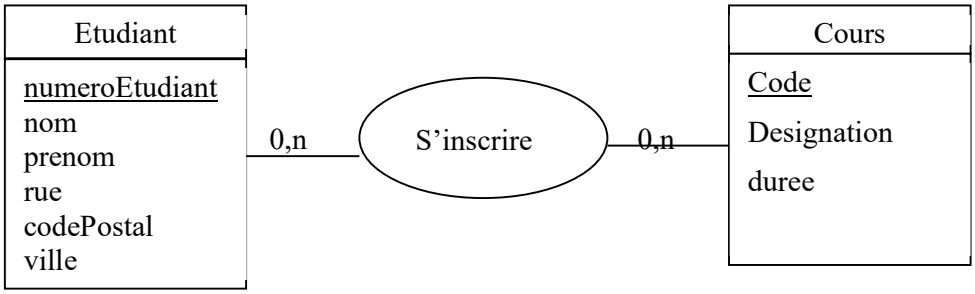
L'identifiant d'une entité est une propriété ou un ensemble de propriétés de cette entité. La valeur de l'identifiant doit être stable, non nulle et caractériser de façon unique l'occurrence correspondante. Il peut être naturel ou artificiel.
Représentation : Pour distinguer l'identifiant des autres propriétés on le souligne et le met en tête.



L'identifiant d'une association est la concaténation des identifiants des entités reliés.
Par définition toute valeur d'identifiant est unique.

f) Les cardinalités

Cardinalité (minimum, maximum): exprime le nombre de fois où chaque occurrence d'entité participe à la relation.
Les cardinalités d'une entité vis-à-vis d'une association indique le nombre minimum(0 ou 1) et maximum(1 ou n) d'occurrence de l'association auxquelles participe chaque occurrence de l'entité. Les cardinalités sont écrites au bout de chaque patte de l'association à côté de l'entité correspondante. Les cardinalités maximum et minimum sont séparés par une virgule.



Certaines cardinalités minimums sont négociables dans ce cas on peut utiliser 0 ou 1.
Les différentes cardinalités possibles sont (0,1),(1,1),(0,n) et (1,n).
Les cardinalités maximums sont très utiles pour le passage vers le MLD. Ainsi une association peut être désigné par ces cardinalités maximums.
Une association [1-n] est une binaire dont une cardinalité maximale vaut 1 et l'autre n.
Une association [n-n] est une association dont toutes les cardinalités maximales valent n.
Comment construire un MCD :

- Ne tenez pas compte dans le sujet de ce qui a trait au traitement et de ce qui est extérieur au domaine étudié.

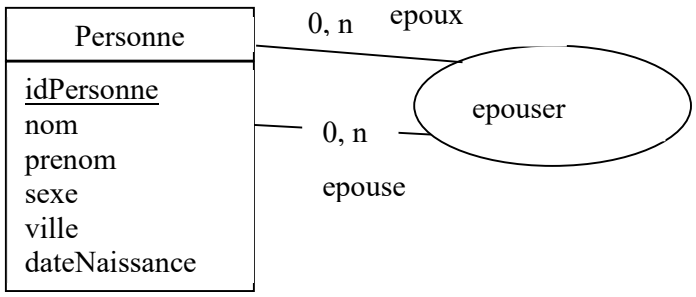
- Identifiez les entités et les associations.
- Identifiez les propriétés.
- Déterminez les identifiants naturels et y ajoutez les artificiels.
- Déterminez le MCD, pour chaque association placez les cardinalités.

g) Les associations porteuses de données

Une association est porteuse de données si elle possède une ou plusieurs propriétés. L'identifiant de l'association déterminant l'occurrence de l'association, il déterminera aussi la valeur de ces propriétés.

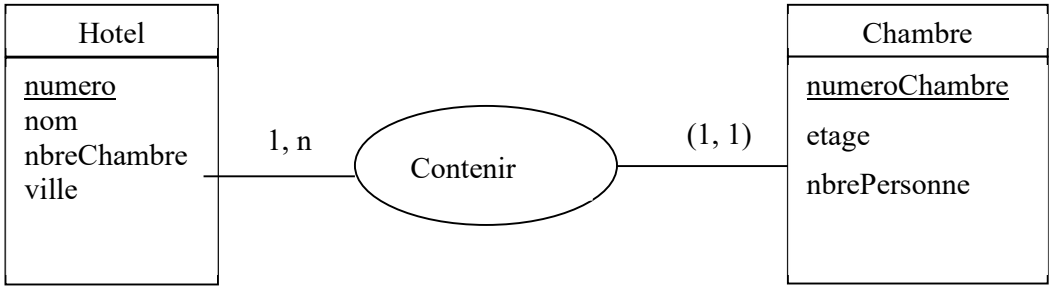
h) Les associations réflexives

Une association est réflexive si une entité participe plus d'une fois à cette association. Il est préférable de toujours indiquer le rôle de chaque patte et il faut bien vérifier les cardinalités.



i) Les entités faibles

Une entité faible est une entité qui possède une existence propre dépendant de l'existence d'une autre entité.



Exercice 1.1

La direction du College De Paris souhaite informatiser son activité. Une analyse détaillée de l'organisation actuelle fait apparaitre que CDP propose des inscriptions à des formations décrites comme suit.

Une formation possède un titre (par exemple 1ere année Licence Informatique), un niveau initial requis et une durée en mois. La formation est composée des modules (cours). Par exemple la formation 1ere année Licence comprend les modules français, anglais, Merise, Algorithme…

Un module possède un titre, un nombre de devoirs, une durée en heure et un coefficient vis-à-vis de la formation. Il faut noter qu'un module n'appartient qu'à une formation. En effet, les programmes n'étant pas les mêmes, le module de français 1ere année Informatique n'est pas le même que celui de la 3eme année Economie.

Un étudiant (nom, prénom, adresse) s'inscrit à des formations. Il peut arriver qu'un étudiant suive plusieurs formations. Pour chacune d'elles, l'étudiant aura une moyenne.

Faites le MCD modélisant les formations.

2) Le modèle logique de données (MLD)

Le MLD est une traduction du MCD, où l'on exprime une solution encore assez générale, permettant une portabilité ultérieure, sans remise en question fondamentale de l'architecture des données. C'est ainsi qu'il sera lui aussi indépendant du matériel et du logiciel, il ne fait que prendre en compte l'organisation des données.
Le MLD est l'expression du MCD dans le Système de Gestion des Bases de Données (SGBD) relationnel.

- Les entités du MCD seront alors converties en tables dans le MLD et selon les cardinalités, les associations sont convertis en tables ou supprimées.
- Le vocabulaire fait appel à des termes différents qui se réfèrent aux mêmes concepts.

Vocabulaire MCD	**Vocabulaire MLD et SGBD**
identifiant	Clé primaire
propriété	champ

Il existe un concept propre au MLD qui est la clé étrangère. **Une clé étrangère est un champ d'une table qui est clé primaire dans une autre table.**

Rappel

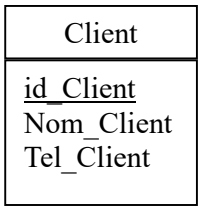
Dans une base de données l'attribut qui permet d'identifier de façon unique chaque ligne est appelée la **Clé Primaire**. Elle peut être composée, c'est à dire comprendre plusieurs attributs. On appel **Clé Etrangère** l'attribut provenant d'une autre table. Dans le formalisme, la clé primaire est soulignée, et la clé étrangère est précédée du signe **#.**
MATABLE (Cle_Primaire, Colonne1, Colonne2, #Cle_Etrangere)

a) Règles de passage du MCD au MLD

Dans cette partie nous définirons 6 règles strictes, nécessaires et suffisantes pour passer d'un MCD à un MLD.

Règle 1 : Une entité se transforme en une table

Toute entité du MCD devient une relation du MLDR, et donc une table de la Base de Données. Chaque propriété de l'entité devient un champ de cette table, et donc une colonne de la table correspondante. L'identifiant de l'entité devient la **Clé Primaire** de la relation (elle est donc soulignée), et donc la **Clé Primaire** de la table correspondante.



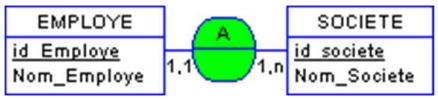
<==> CLIENT (id_Client, Nom_Client, Tel_Client)

Règle 2 : association [1,n] donc aux cardinalités (X,1) - (X,n) avec X=0 ou X=1

La **Clé Primaire** de la table à la cardinalité (X,n) devient une **Clé Etrangère** dans la table à la cardinalité (X,1) .

Exemple de Système d'Information (SI) :
Un employé a une et une seule société. Une société a 1 ou n employés.

MCD :

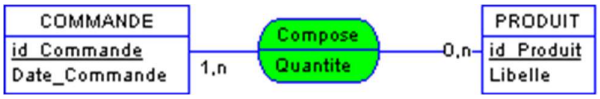


MLDR:
EMPLOYE (id_Employe, Nom_Employe, #id_Societe)
SOCIETE (id_Societe, Nom_Societe)

Règle 3 : Relation binaire [n,n] aux cardinalités (X,n) - (X,n), X=0 ou X=1

Il y a création d'une table supplémentaire ayant comme **Clé Primaire** une clé composée des **identifiants** des 2 entités. On dit que la **Clé Primaire** de la nouvelle table est la **concaténation** des **Clés Primaires** des deux autres tables. Si la relation est porteuse de donnée, celles ci deviennent des champs pour la nouvelle table.
SI:
Une commande est composée de 1 ou n produits distincts en certaine quantité. Un produit est présent dans 0 ou n commandes en certaine quantité.

MCD :

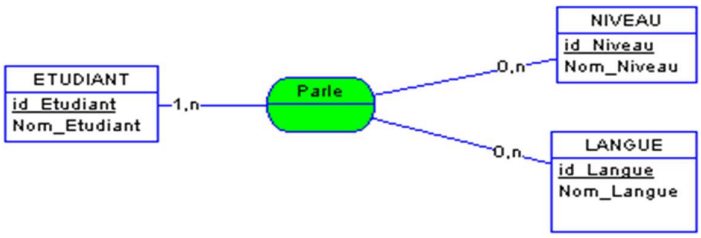


MLDR :
COMMANDE(id_Commande,Date_commande)
PRODUIT(id_Produit,libelle)
COMPOSE (id_Commande, id_Produit, qantité)

Règle 4 : Association n-aire donc [n,n].

Il y a création d'une table supplémentaire ayant comme **Clé Primaire** la **concaténation** des **identifiants** des entités participant à l'association. Si la relation est porteuse de donnée, celles ci deviennent des attributs pour la nouvelle table.

SI :
Un étudiant parle une ou plusieurs langues avec un niveau. Chaque langue est donc parlée par 0 ou n étudiants avec un niveau. Pour chaque niveau, il y a 0 ou plusieurs étudiants qui parlent une langue.
MCD :



MLDR :
ETUDIANT (id_Etudiant,Nom_Etudiant)
NIVEAU (id_Niveau,Nom_Niveau)
LANGUE (id_Langue,Nom_Langue)
PARLE (id_Etudiant, id_Niveau, id_Langue)

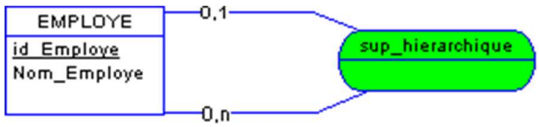
Règle 5 : Association Réflexive.

- Premier cas : cardinalité (X,1) - (X,n), avec X=0 ou X=1.

La **Clé Primaire** de l'entité se dédouble et devient une **Clé Etrangère** dans la relation ou nouvelle table. Exactement comme si l'entité se dédoublait et était reliée par une relation binaire (X,1) - (X,n) (Cf règle 2).

SI:
Prenons l'exemple d'une société organisée de manière pyramidale : chaque employé a 0 ou 1 supérieur hiérarchique direct. Simultanément, chaque employé est le supérieur hiérarchique direct de 0 ou plusieurs employés.

MCD :



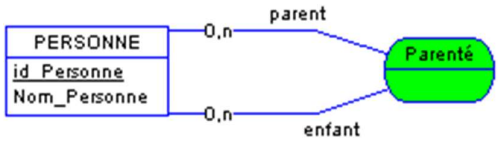
MLDR :
EMPLOYE (id_Employe, Nom_Employe, #id_Sup_Hierarchique)
#id_Sup_Hierarchique est l'identifiant (id_Employe) du supérieur hiérarchique direct de l'employé considéré.

- Deuxième cas : cardinalité (X,n) - (X,n), avec X=0 ou X=1.

De même, tout se passe exactement comme si l'entité se dédoublait et était reliée par une relation binaire (X,n) - (X,n) (Cf règle 3). Il y a donc création d'une nouvelle table.

SI:
Prenons cette fois l'exemple d'une organisation de type familiale : chaque personne a 0 ou n ascendants directs (parents), et a aussi 0 ou n descendants directs (enfants).

MCD :



MLDR :
PERSONNE (id_Personne, Nom_Personne)
PARENTE (#id_Parent, #id_Enfant)
#id_Parent est l'identifiant (id_Personne) d'un ascendant direct de la personne. #id_Enfant est l'identifiant (id_Personne) d'un descendant direct de la personne. La table PARENTE sera en fait l'ensemble des couples (parents-enfants) présent dans cette famille.

Règle 6 : Relation binaire aux cardinalités (0,1) - (1,1).

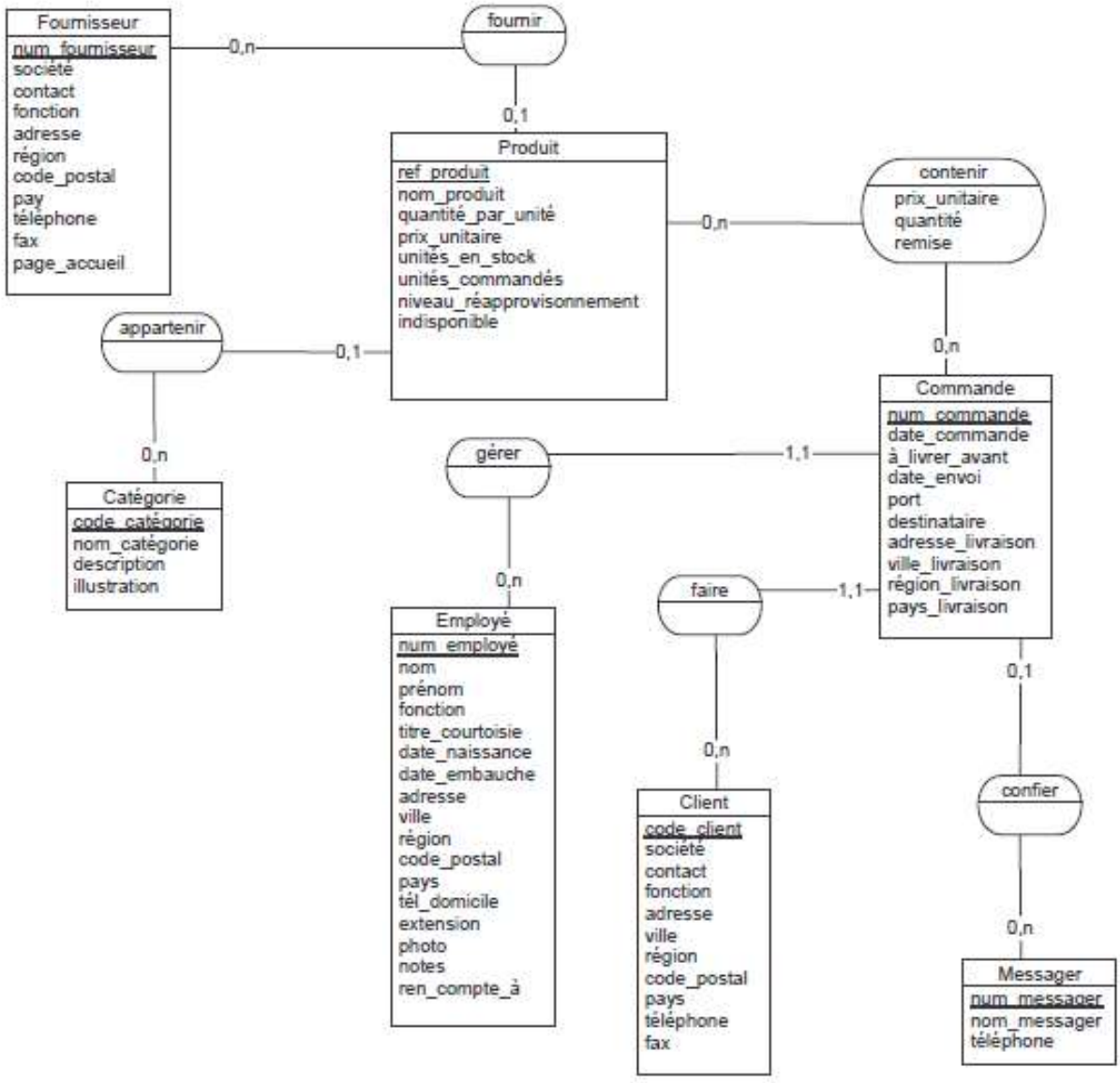
La **Clé primaire** chaque table devient **Clé Etrangère** dans l'autre table. Puis on assure l'unicité de ces clés étrangères.

SI:
Dans un centre de vacances, chaque animateur encadre en solo 0 ou 1 groupe, chaque groupe étant encadré par un et un seul animateur.



MLDR :
ANIMATEUR (id_Animateur, Nom_Animateur, #id_Groupe)
GROUPE (id_Groupe, Nom_Groupe, #id_Animateur)

Exercice 1.2 *Donner le Modèle Logique des données du MCD ci-dessous*



II. Normalisation (1NF, 2NF, 3NF, BCNF)

a) Les dépendances fonctionnelles :

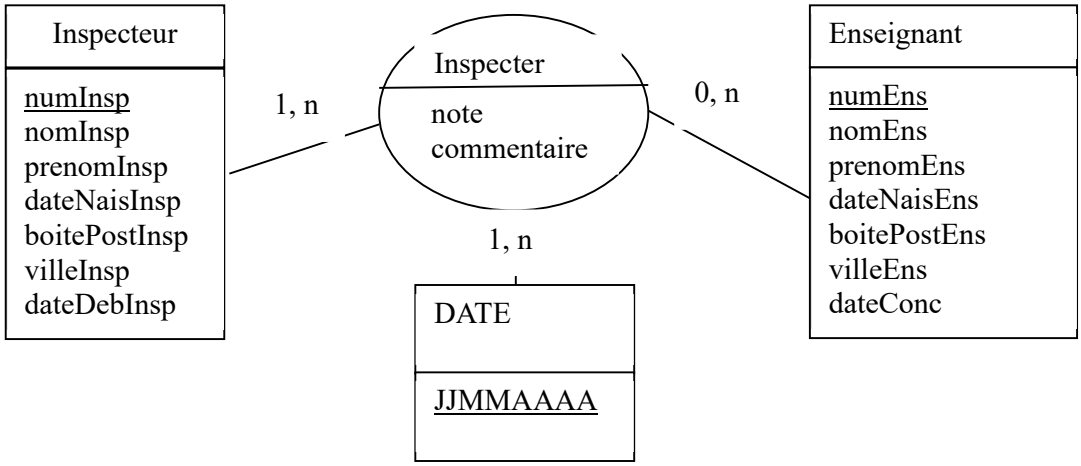
On dira qu'une propriété Q dépend fonctionnellement de la propriété P(ou des propriétés P1,P2,..,Pn) si la valeur de P(P1,P2,..,Pn) détermine celle de Q, donc si à une valeur de P(P1,P2,..,Pn) correspond une unique valeur de Q.
Exemple : les autres propriétés d'une entité dépendent fonctionnellement de l'identifiant.

- Dépendance fonctionnelle (DF) intra-entité (entre deux propriétés de la même entité) :
  - DF (p1, p2) : p1➔ p2
  - A toute valeur de p1, on ne peut associer à tout instant qu'une et une seule valeur de p2
  - *« Si on connaît la valeur de p1, on connaît à coup sûr la valeur de p2 »*
  - exemple : numClient ➔ NomClient (la réciproque est fausse)
- Dépendance fonctionnelle (DF) inter-entités (entre deux entités):
  - DF (E1, E2) : E1➔ E2
  - A toute occurrence de E1, on ne peut associer à tout instant qu'une et une seule occurrence de E2

« Si on connaît la valeur de id11 (identifiant de E1), on connaît à coup sûr la valeur de id21 (identifiant de E2)».

Exercice 1.3

Dans le MCD suivant donnez toutes les dépendances fonctionnelles. Il y en a 14.



1) Présentation

La normalisation des bases de données est un processus de conception qui organise les données dans des tables pour minimiser la redondance et améliorer l'intégrité des données.

La normalisation joue un rôle crucial dans la conception des bases de données. Voici plusieurs raisons pour lesquelles il est essentiel :

- **Réduit la redondance :** On parle de redondance lorsque la même information est stockée plusieurs fois. Un bon moyen d'éviter cela est de diviser les données en tableaux plus petits.
- **Améliore les performances des requêtes :** Vous pouvez exécuter plus rapidement des requêtes sur des tableaux plus petits ayant fait l'objet d'une normalisation.
- **Minimise les anomalies de mise à jour :** Avec les tableaux normalisés, vous pouvez facilement mettre à jour les données sans affecter les autres enregistrements.
- **Améliore l'intégrité des données :** Il garantit la cohérence et l'exactitude des données.

2) La première forme normale (1NF)

L'entité ou l'association est en 1ere FN si toutes les propriétés sont élémentaires et dépendent fonctionnellement de l'identifiant.
Ce niveau de normalisation garantit que chaque colonne de vos données ne contient que des valeurs atomiques. Dans ce contexte, les valeurs atomiques signifient que chaque entrée d'une colonne est indivisible. La 1NF garantit l'atomicité des données, chaque cellule de colonne ne contenant qu'une seule valeur et chaque colonne ayant un nom unique.
Les tables ne doivent pas contenir des groupes de données répétitives. Si tel est le cas, il faut sortir le groupe de données et créer une autre entité qui va contenir ce groupe de données.

3) La deuxième forme normale (2NF)

L'entité ou l'association est en 2e FN si elle est en 1ere FN et, toute propriété non identifiant est en DF élémentaire (une propriété cible et une source minimale) avec l'identifiant.
Élimine les dépendances partielles en veillant à ce que les attributs autres que la clé ne dépendent que de la clé primaire.
Une entité (Table) ou une relation est en deuxième forme normale 2FN, si elle est :

- En 1FN
- Tous les attributs de la relation ou de l'entité dépendent de toute la clé (notion de clé primaire composée) et non d'une partie de la clé

4) La troisième forme normale (3NF)

Supprime les dépendances transitives en veillant à ce que les attributs non clés ne dépendent que de la clé primaire. Ce niveau de normalisation s'appuie sur la norme 2NF.
Une entité ou une relation est en troisième forme normale (3FN) si

- Elle est déjà en 2FN
- Tous les attributs non clé dépendent uniquement de la clé. Il n'y a pas de dépendance entre deux attributs non clé.

En résumé une entité ou une association doit être en 3e FN, à savoir : les propriétés élémentaires dépendent de l'identifiant (1ere FN), de tout l'identifiant (2e FN) et que de l'identifiant (3e FN).

5) La forme normale de Boyce Codd (BCNF)

Il s'agit d'une version plus stricte de la 3NF qui tient compte d'autres anomalies. À ce niveau de normalisation, chaque déterminant est une clé candidate.

Une relation (table) est en BCNF si

- Elle est déjà en 3FN
- Les seules dépendances fonctionnelles sont celles dans lesquelles une clé (entière si composée) détermine un attribut.

III. Concepts de tables, clés, relations, intégrité référentielle

1) Concepts de tables

L'unité de stockage élémentaire dans une base de données relationnelle est la « table ». En général, c'est aux tables que les utilisateurs font référence pour accéder aux données. Une base de données est constituée de plusieurs tables reliées entre elles.
Une table est constituée de lignes et de colonnes.
Exemple de table Livres qui contient des informations des livres de notre bibliothèque.

NumeroLivre	TitreLivre	Auteur	Langue
101	Ainsi parlait Zarathoustra	Friedrich Nietzsche	Allemand
102	La paix des profondeurs	Aldous Huxley	Anglais
103	Les misérables	Victor Hugo	Français
104	La liberté n'est pas une marque de yogourt	Pierre Falardeau	Françiais

2) Relations

Une relation dans une table est une association entre deux tables qui permet de relier des données en faisant correspondre une clé **primaire** d'une table à une **clé étrangère** d'une autre. Il existe plusieurs types de relations :

- **Un-à-un** : Un enregistrement dans la table A ne peut être associé qu'à un seul enregistrement dans la table B, et vice-versa.
  - *Exemple* : Chaque employé est associé à une seule voiture de service.
- **Un-à-plusieurs** : Un enregistrement dans la table A peut être associé à plusieurs enregistrements dans la table B, mais chaque enregistrement dans la table B ne peut être associé qu'à un seul enregistrement dans la table A.
  - *Exemple* : Un bon de commande (table A) peut contenir plusieurs produits (table B).
- **Plusieurs-à-plusieurs** : Un enregistrement dans la table A peut être associé à plusieurs enregistrements dans la table B, et vice-versa.
  - *Exemple* : Une commande peut contenir plusieurs produits, et chaque produit peut être présent dans plusieurs commandes différentes

3) Clés

- **Clé primaire** : Un champ dans une table qui identifie de manière unique chaque enregistrement de cette table.
- **Clé étrangère** : Un champ dans une table qui correspond à la clé primaire d'une autre table, créant ainsi un lien.

4) Intégrité référentielle

L'intégrité référentielle est une règle de base de données qui garantit la cohérence des relations entre des tables. Elle assure que les valeurs des clés étrangères d'une table sont toujours valides, c'est-à-dire qu'elles correspondent à une clé primaire existante dans une autre table, ou sont nulles. Sa mise en œuvre se fait par :

- **Lien entre les tables :** Une clé étrangère dans une table (par exemple, ID_Client dans la table Commandes) fait référence à la clé primaire d'une autre table (par exemple, ID_Client dans la table Clients).
- **Règles de cohérence :** La contrainte d'intégrité référentielle est appliquée pour empêcher des actions qui tendent à rompre le lien :
  - Supprimer une ligne dans la table parent (table Clients) si des enregistrements correspondants existent dans la table enfant (table Commandes).

    - Modifier une clé primaire dans la table parent si des enregistrements correspondants existent dans la table enfant.
- **Actions possibles :** on peut définir des actions automatiques :
    - **Cascade :** Si une ligne est supprimée ou modifiée dans la table parent, les lignes correspondantes dans la table enfant sont automatiquement supprimées ou modifiées.
    - **Restrict (par défaut) :** Une erreur est générée si l'opération de suppression ou de modification est tentée et qu'il y a des correspondances.
    - **Set Null :** Si la clé étrangère le permet, la valeur de la clé étrangère peut être automatiquement mise à NULL.
- **Avantages**

- **Prévient les incohérences :** Empêche l'existence de données orphelines et garantit des relations valides entre les enregistrements.
- **Maintient l'exactitude des données :** Assure que les données restent cohérentes et précises au fil du temps.
- **Favorise la traçabilité :** Améliore la traçabilité des données, réduisant les risques de perte de données sensibles

CHAPITRE 2 : Langage SQL Avancé

I. Présentation du SQL

SQL signifie :Structured Query Language c'est-à-dire Langage d'interrogation structuré.
En fait SQL est un langage complet de gestion de bases de données relationnelles. Il a été conçu par IBM dans les années 70. Il est devenu le langage standard des systèmes de gestion de bases de données (SGBD) relationnelles (SGBDR).
C'est à la fois :

- un langage d'interrogation (ordre SELECT),
- un langage de manipulation des données (ordres UPDATE, INSERT, DELETE),
- un langage de définition des données (LDD ; ordres CREATE, ALTER, DROP),
- un langage de contrôle de l'accès aux données (LCD ; ordres GRANT, REVOKE).

II. Requêtes conditionnelles et expressions CASE

1) Requêtes conditionnelles

Les instructions conditionnelles SQL te permettent d'effectuer des opérations basées sur des conditions spécifiques, qui impliquent souvent de comparer des valeurs, de filtrer des enregistrements et de manipuler des données en fonction d'exigences spécifiques. Cela fait des instructions conditionnelles SQL un outil essentiel pour travailler avec des bases de données relationnelles. Il s'agit des déclarations en SQL qui permettent d'effectuer des opérations en fonction de conditions spécifiées.

a) Importance des instructions conditionnelles SQL

L'utilisation des instructions conditionnelles SQL a un impact important sur la gestion et la récupération des données. Ces instructions permettent de :

- Filtrer et renvoyer des enregistrements spécifiques en fonction de critères précis ;
- Créer et mettre à jour des données à condition que des critères spécifiques soient remplis ;
- Contrôler le flux des requêtes SQL pour effectuer des opérations complexes de manière efficace ;
- Implémenter des structures de contrôle, telles que la logique if-then-else, dans les requêtes SQL ;
- Améliorer les performances de la base de données en limitant les résultats des requêtes aux enregistrements pertinents.

b) Types courants d'instructions conditionnelles SQL

Il existe plusieurs types courants d'instructions conditionnelles SQL :

Opérateurs de comparaison	=, <>, !=, <,>, <= et >=
Opérateurs logiques	AND, OR et NOT
BETWEEN	Utilisé pour sélectionner des valeurs dans une plage spécifiée
LIKE...	Utilisé pour rechercher un modèle spécifié
IN... NOT IN	Permet de faire correspondre une valeur à un ensemble de valeurs
CASE...	Utilisé pour effectuer la logique if-then-else en SQL

c) Déclarations conditionnelles SQL dans la clause WHERE

La clause WHERE en SQL est l'endroit où on utilise souvent les instructions conditionnelles. La clause WHERE filtre l'ensemble des résultats en spécifiant les conditions qui doivent être remplies par les enregistrements. Par exemple :

SELECT * FROM customers WHERE country = 'TG' ;

Cette requête SQL renvoie tous les enregistrements de la table clients dont le pays est 'TG'. Les instructions conditionnelles de la clause WHERE peuvent être combinées à l'aide d'opérateurs logiques, comme illustré ci-dessous :

SELECT * FROM customers WHERE country = 'TG' AND age >= 18 ;

Dans cet exemple, la requête renvoie tous les enregistrements de la table customers où le pays est 'TG' et l'âge est supérieur ou égal à 18.

d) Les instructions conditionnelles SQL dans SELECT

En plus d'utiliser les instructions conditionnelles SQL dans la clause WHERE, on peut aussi les utiliser dans la clause SELECT. Cette approche permet de manipuler les données à la volée tout en les récupérant dans la base de données. Pour comprendre les instructions conditionnelles SQL dans les clauses SELECT, il faut savoir comment utiliser les expressions CASE, ainsi que les fonctions d'agrégation, telles que COUNT, SUM et AVG, avec les instructions conditionnelles SQL.

2) Expressions CASE

a) Qu'est-ce que la commande CASE ?

L'expression SQL CASE permet d'effectuer une logique conditionnelle dans une instruction SELECT. Il s'agit essentiellement d'un mécanisme permettant de définir des instructions de type if-then-else qui peuvent être utilisées pour créer, mettre à jour ou effectuer des calculs en fonction de critères spécifiques. Il permet de **retourner un résultat parmi plusieurs possibilités.**

Selon la syntaxe utilisée, CASE permet de :

- Comparer une donnée à un set de résultat possible. Dans ce cas, voici la syntaxe de CASE SQL :

```
CASE
WHEN expression THEN résultat
ELSE autre résultat
END
```

- Déterminer un résultat en élaborant plusieurs expressions booléennes (vrai ou faux). La syntaxe est la suivante :

```
CASE
WHEN expression booléenne THEN résultat
ELSE autre résultat
END
```

b) Exemples d'utilisation de CASE SQL

Afin de vous aider à mieux comprendre l'utilisation de CASE SQL, voici plusieurs exemples concrets.

Pour chacun d'entre eux, nous allons utiliser ce tableau « Achat ». L'objectif de l'entreprise est de définir le bon prix. Pour cela, elle se base sur le taux de marge réalisé.

ID	Nom du produit	Marge	Quantité	Prix unitaire
1	Produit 1	1	1	50
2	Produit 2	1,25	3	70
3	Produit 3	0,75	2	60

ID	Nom du produit	Marge	Quantité	Prix unitaire
4	Produit 4	1,5	2	80

Message personnalisé

L'objectif est d'informer les décisionnaires sur le positionnement des prix de chaque produit. Pour cela, il est possible d'utiliser la requête CASE SQL.

Voici la syntaxe :

```
SELECT id, nom du produit, marge,quantité, prix unitaire,
   CASE
     WHEN marge > 1 THEN 'Prix élevé'
     WHEN marge < 1 THEN 'Prix bas'
     ELSE 'Prix normal'
   END
FROM `Achat`
```

Et le résultat :

ID	Nom du produit	Marge	Quantité	Prix unitaire	CASE
1	Produit 1	1	1	50	Prix normal
2	Produit 2	1,25	3	70	Prix élevé
3	Produit 3	0,75	2	60	Prix bas
4	Produit 4	1,5	2	80	Prix élevé

Modification du prix unitaire

Afin d'ajuster sa politique tarifaire, l'entreprise souhaite modifier ses prix en fonction de sa marge : réduire de 10 % lorsque la marge est positive et augmenter de 10 %, lorsqu'elle est négative.

Là encore, l'utilisation des conditions et de la commande CASE SQL permettent de modifier rapidement la base de données.

Voici la syntaxe :

```
SELECT id, nom du produit, marge,quantité, prix unitaire,
   CASE
     WHEN marge > 1 THEN 'Prix unitaire * 0,9'
     WHEN marge < 1 THEN 'Prix unitaire * 1,1'
     ELSE 'Prix unitaire'
   END
FROM `Achat`
```

Et le résultat :

ID	Nom du produit	Marge	Quantité	Prix unitaire	CASE
1	Produit 1	1	1	50	50
2	Produit 2	1,25	3	63	56,7
3	Produit 3	0,75	2	66	72,6
4	Produit 4	1,5	2	72	64,8

Actualisation de la quantité

Au-delà de la SELECT, la requête CASE SQL peut aussi être utilisée avec d'autres commandes, comme UPDATE.

Afin de vous aider à visualiser cette requête, voici un scénario hypothétique.

Pour tous les produits présentant une marge positive, l'entreprise offre un autre produit supplémentaire. Dans ce cadre, il faudra mettre à jour le tableau.

Voici comment faire :

```
UPDATE `achat`
SET `quantité` = (
   CASE
     WHEN marge > 1 THEN 'Quantité +  1'
     ELSE 'Quantité'
   END
)
```

Et le résultat :

ID	Nom du produit	Marge	Quantité	Prix unitaire
1	Produit 1	1	1	50
2	Produit 2	1,25	4	70
3	Produit 3	0,75	2	60
4	Produit 4	1,5	3	80

III. Requêtes complexes et les vues

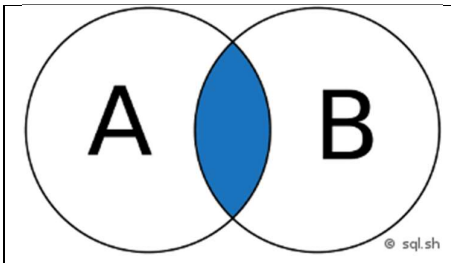
Les requêtes SQL complexes combinent plusieurs commandes et conditions pour extraire ou manipuler des données, notamment grâce à l'utilisation de **jointures** et de **sous-requêtes**. Elles permettent de lier des données de plusieurs tables, d'effectuer des calculs complexes, et de filtrer les résultats selon des critères multiples.

1) Les jointures

Les jointures en SQL permettent d'associer plusieurs tables dans une même requête. Cela permet d'exploiter la puissance des bases de données relationnelles pour obtenir des résultats qui combinent les données de plusieurs tables de manière efficace. En général, les jointures consistent à associer des lignes de 2 tables en associant l'égalité des valeurs d'une colonne d'une première table par rapport à la valeur d'une colonne d'une seconde table. Imaginons qu'une base de données possède une table "utilisateur" et une autre table "adresse" qui contient les adresses de ces utilisateurs. Avec une jointure, il est possible d'obtenir les données de l'utilisateur et de son adresse en une seule requête.

Il y a plusieurs méthodes pour associer 2 tables ensemble. Voici la liste des différentes techniques qui sont utilisées :

- **INNER JOIN :** jointure interne pour retourner les enregistrements quand la condition est vrai dans les 2 tables. C'est l'une des jointures les plus communes.



Intersection de 2 ensembles

```
SELECT *
FROM A
INNER JOIN B ON A.key = B.key
```

Considérons les tables Film et Acteurs suivantes :

La table Film

ID_FILM	NOM_DU_FILM	ANNÉE_DE_SORTIE	NOTE_IMDB
1	Jaws	1975	8.0
2	Star Wars	1977	8.6
3	Don't Look Up	2021	7.2
4	Titanic	1997	7.8
5	Le dernier Samouraï	2003	7.7

La table Acteur

NOM_ACTEUR	*DATE_DE_NAISSANCE*	*ID_FILM*
Roy Scheider	*10/11/1932*	*1*
Mark Hamill	*25/09/1951*	*2*
Meryl Streep	*22/09/1949*	*3*
Tom Cruise	*03/07/1962*	*5*
Tom Hanks	*15/09/1977*	*NULL*
Timothée Chalamet	*27/12/1995*	*3*

```
SELECT ID_film, Nom_du_film, Annee_de_sortie, Note_IMDB
FROM Film
INNER JOIN Acteur
ON Cinema.ID_film = Acteur.ID_film
```

ID_film	Nom_du_film	Année_de_sortie	Note_IMDb	Nom_Acteur	Date_de_naissance
1	Jaws	1975	8.0	Roy Scheider	10/11/1932
2	Star Wars	1977	8.6	Mark Hamill	25/09/1951
3	Don't Look Up	2021	7.2	Meryl Streep	22/09/1949
3	Don't Look Up	2021	7.2	Timothée Chalamet	27/12/1995
5	Le dernier Samouraï	2003	7.7	Tom Cruise	03/07/1962

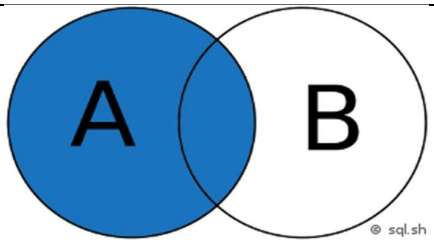
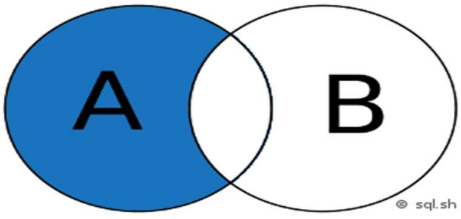
- **CROSS JOIN :** jointure croisée permettant de faire le produit cartésien de 2 tables. En d'autres mots, permet de joindre chaque lignes d'une table avec chaque lignes d'une seconde table. Attention, le nombre de résultats est en général très élevé.

```
SELECT *
FROM acteurs
```

CROSS JOIN cinema

ID_film_Cinema	Nom_du_film	Année_de_sortie	Note_IMDb	Nom_Acteur	Date_de_naissance	ID_film_Acteur
1	Jaws	1975	8.0	Roy Scheider	10/11/1932	1
1	Jaws	1976	8.1	Mark Hamill	25/09/1951	2
1	Jaws	1977	8.2	Meryl Streep	22/09/1949	3
1	Jaws	1978	8.3	Timothée Chalamet	27/12/1995	3
1	Jaws	1980	8.5	Tom Cruise	03/07/1962	5
1	Jaws	1981	8.6	Tom Hanks	15/09/1977	NULL
2	Star Wars	1977	8.6	Roy Scheider	10/11/1932	1
2	Star Wars	1977	8.6	Mark Hamill	25/09/1951	2
2	Star Wars	1977	8.6	Meryl Streep	22/09/1949	3
2	Star Wars	1977	8.6	Timothée Chalamet	27/12/1995	3
2	Star Wars	1977	8.6	Tom Cruise	03/07/1962	5
2	Star Wars	1977	8.6	Tom Hanks	15/09/1977	NULL

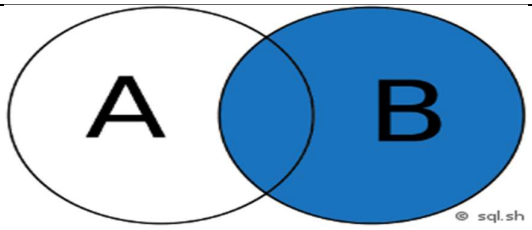
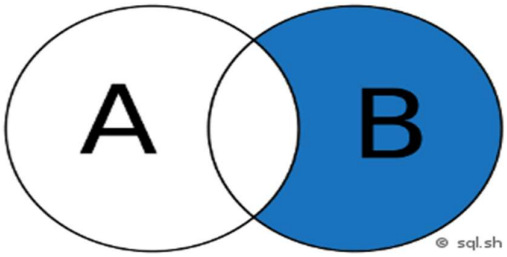
- **LEFT JOIN (ou LEFT OUTER JOIN) :** jointure externe pour retourner tous les enregistrements de la table de gauche (LEFT = gauche) même si la condition n'est pas vérifié dans l'autre table.

	SELECT * FROM A LEFT JOIN B ON A.key = B.key
LEFT JOIN (sans l'intersection de B) 	Jointure gauche (LEFT JOINT sans l'intersection B) SELECT * FROM A LEFT JOIN B ON A.key = B.key WHERE B.key IS NULL

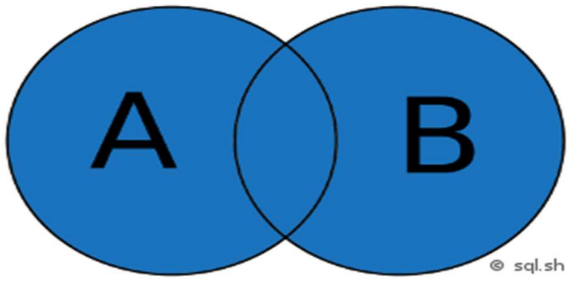
*SELECT **
FROM cinema
LEFT JOIN acteurs
ON cinema.id_film = acteurs.id_film

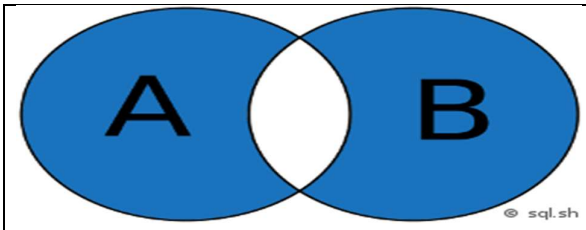
ID_film	Nom_du_film	Année_de_sortie	Note_IMDb	Nom_Acteur	Date_de_naissance
1	Jaws	1975	8.0	Roy Scheider	10/11/1932
2	Star Wars	1977	8.6	Mark Hamill	25/09/1951
3	Don't Look Up	2021	7.2	Meryl Streep	22/09/1949
3	Don't Look Up	2021	7.2	Timothée Chalamet	27/12/1995
4	Titanic	1997	7.8	NULL	NULL
5	Le dernier Samouraï	2003	7.7	Tom Cruise	03/07/1962

- **RIGHT JOIN (ou RIGHT OUTER JOIN) :** jointure externe pour retourner tous les enregistrements de la table de droite (RIGHT = droite) même si la condition n'est pas vérifié dans l'autre table.

	Jointure droite (RIGHT JOINT) SELECT * FROM A RIGHT JOIN B ON A.key = B.key
RIGHT JOIN (sans l'intersection de A) 	Jointure droite (RIGHT JOINT sans l'intersection A) SELECT * FROM A RIGHT JOIN B ON A.key = B.key WHERE B.key IS NULL

- **FULL JOIN (ou FULL OUTER JOIN) :** jointure externe pour retourner les résultats quand la condition est vrai dans au moins une des 2 tables.

	Union de 2 ensembles SELECT * FROM A FULL JOIN B ON A.key = B.key
FULL JOIN (sans intersection)	Jointure pleine (FULL JOINT sans intersection) SELECT *

	FROM A FULL JOIN B ON A.key = B.key WHERE A.key IS NULL OR B.key IS NULL

- **SELF JOIN :** permet d'effectuer une jointure d'une table avec elle-même comme si c'était une autre table. Le SELF JOIN est une jointure qui est faite **sur elle-même**. Cela revient à réaliser une jointure sur **deux tables identiques**. Ces jointures sont utilisées lorsqu'une table possède une clé primaire et une clé étrangère à la fois**.**
- **NATURAL JOIN :** jointure naturelle entre 2 tables s'il y a au moins une colonne qui porte le même nom entre les 2 tables SQL
- **UNION JOIN :** jointure d'union

2) Les Sous-requêtes

Une sous-requête permet aux requêtes SQL d'être plus modulaires en traitant des tâches qui nécessiteraient autrement plusieurs requêtes distinctes.

a) Définition et objectif

Une sous-requête SQL est une requête imbriquée dans une autre requête SQL, utilisée pour effectuer des opérations nécessitant plusieurs étapes ou une logique complexe. Le rôle des sous-requêtes dans SQL est notamment le suivant :

- Filtrage des enregistrements sur la base de données provenant de tableaux connexes ;
- Agréger des données et effectuer des calculs de manière dynamique ;
- Recouper les données entre les tableaux pour obtenir des informations spécifiques ;
- Sélection conditionnelle des lignes sans nécessiter de jointures explicites ou de code logique externe.

b) Syntaxe

La syntaxe d'une sous-requête varie en fonction de l'endroit où elle est utilisée dans l'instruction SQL principale, par exemple dans les clauses `SELECT`, `FROM` ou `WHERE`. Les sous-requêtes sont généralement placées entre parenthèses à l'adresse `( )`, ce qui indique qu'il s'agit d'une requête distincte.

L'exemple suivant montre une sous-requête dans une clause `WHERE`, ce qui nous permet de filtrer les données de la requête principale sur la base des résultats d'une requête imbriquée.

```
-- Selects the main column to retrieve from the main table to query
SELECT column_name
FROM table_name
-- Applies a condition to filter rows based on the subquery result
WHERE column_name operator
      -- Subquery retrieves data for comparison in the WHERE clause
      (SELECT column_name FROM table_name WHERE condition);
```

c) Types de sous-requêtes

Il existe différents types de sous-requêtes. Les différents types sont regroupés en fonction des différents types de besoins en matière de recherche de données et sont adaptés à ceux-ci.

- ***Sous-requêtes scalaires***

Les sous-requêtes scalaires renvoient une seule valeur, par exemple une ligne et une colonne. Ils sont souvent utilisés lorsqu'une seule valeur est attendue, par exemple dans les calculs, les comparaisons ou les affectations dans les clauses `SELECT` ou `WHERE`.
Dans l'exemple ci-dessous, la sous-requête scalaire `(SELECT AVG(salary) FROM employees)` renvoie une seule valeur, le salaire moyen, et la compare au salaire de chaque employé.

```
-- Example of Scalar Subquery
-- Compares each salary to the average salary
SELECT employee_name,
       salary,
       (SELECT AVG(salary) FROM employees) AS average_salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

- ***Sous-requêtes sur les colonnes***

Les sous-requêtes de colonnes renvoient une seule colonne mais plusieurs lignes. Ces sous-requêtes sont souvent utilisées avec des opérateurs tels que IN ou ANY, où la requête externe compare les valeurs de plusieurs lignes.
Par exemple, la sous-requête ci-dessous renvoie une liste d'identifiants de départements situés à Lomé(LM), que la requête principale utilise ensuite pour filtrer les employés de ces départements.

```
-- Example of Column Subquery
-- Filters based on departments in New York
SELECT employee_name
FROM employees
WHERE department_id IN (SELECT department_id FROM departments WHERE location =
'LM';
```

- ***Sous-requêtes de ligne***

Les sous-requêtes de ligne renvoient une seule ligne contenant plusieurs colonnes. Ces sous-requêtes sont généralement utilisées avec des opérateurs de comparaison qui peuvent comparer une ligne de données, tels que les opérateurs = ou IN, lorsque plusieurs valeurs sont attendues.
La sous-requête suivante récupère le service et l'intitulé du poste d'un responsable, et la requête externe recherche les employés dont les valeurs correspondent.

```
-- Example of Row Subquery
-- Matches department and job title with a specific manager
SELECT employee_name
FROM employees
WHERE (department_id, job_title) = (SELECT department_id, job_title FROM managers
WHERE manager_id = 1);
```

- ***Sous-requêtes de tableaux (tableaux dérivés)***

Les sous-requêtes de tableaux, ou tableaux dérivés, renvoient un tableau complet composé de plusieurs lignes et colonnes. Ces tableaux sont généralement utilisés dans la clause FROM en tant que tableau temporaire dans une requête.
Par exemple, la sous-requête ci-dessous crée un tableau dérivé des salaires moyens par département, qui est ensuite utilisé dans la requête externe pour trouver les départements dont le salaire moyen est supérieur à un seuil spécifié.

```
-- Example of Table Subquery
-- Uses derived table for average department salary comparison
```

```
SELECT dept_avg.department_id, dept_avg.avg_salary
FROM
   (SELECT department_id, AVG(salary) AS avg_salary FROM employees GROUP BY
department_id) AS dept_avg
WHERE dept_avg.avg_salary > 50000;
```

3) Les vues

Une **vue** dans une base de données est une synthèse d'une requête d'interrogation de la base. On peut la voir comme une table virtuelle, définie par une requête. C'est une requête nommée.

Les avantages des vues sont :

- d'éviter de taper une requête très longue : la vue sert à donner un nom à la requête pour l'utiliser souvent,
- de masquer certaines données à certains utilisateurs. En SQL, les protections d'une vue ne sont pas forcément les mêmes que celles des tables sous-jacentes.

Les vues se créent avec la commande ***CREATE VIEW***. Un exemple, où les employés d'une entreprise sont dans une table, les départements de l'entreprise dans une autre et où on doit faire une jointure pour afficher le nom du département à côté de celui de l'employé :

```
CREATE TABLE Employes (id SERIAL, nom TEXT, departement INTEGER);
CREATE TABLE Departements (id SERIAL, nom TEXT);
SELECT e.nom as Employe, d.nom as Departement
      FROM Employes e,Departements d WHERE e.departement = d.id;
```

Si, par contre, on crée une vue :

```
CREATE VIEW ToutLeMonde AS
    SELECT e.nom as Employe, d.nom as Departement
        FROM Employes e,Departements d
        WHERE e.departement = d.id;
```

On pourra alors écrire la requête SELECT ci-dessus bien plus simplement, la jointure ne sera plus visible :

```
SELECT * FROM ToutLeMonde ;
```

Les **vues** s'utilisent pratiquement comme des tables (elles peuvent être dans une clause FROM d'un SELECT, dans un UPDATE, etc)

Le résultat de la requête est réévalué à chaque fois que l'on accède à la vue.

IV. Fonctions d'agrégation et fonctions analytiques (OVER, PARTITION)

1) Fonctions d'agrégation

Il s'agit de fonctions spécialisées qui effectuent des calculs sur des groupes de variables et renvoient un résultat unique. Contrairement aux fonctions traditionnelles, les fonctions agrégées travaillent sur des groupes de lignes de données. Cela vous permet de calculer efficacement des statistiques ou de générer des informations récapitulatives à partir d'un ensemble de données. Dans le monde de SQL, les fonctions agrégées sont des outils essentiels pour résumer et analyser efficacement les données. Elles ont la capacité unique de distiller de grands ensembles de données en informations significatives, de faciliter l'analyse statistique et de simplifier les structures de données complexes.

Les fonctions agrégées SQL les plus fréquemment utilisées sont les suivantes :

- SUM()
- COUNT()
- AVG()
- MIN()

- MAX()

a) SUM()

La fonction SUM() renvoie le total d'une colonne numérique. Elle est généralement utilisée lorsque vous devez trouver le total de valeurs telles que le revenu des ventes, les quantités ou les dépenses.

Connaître le revenu total des ventes de votre entreprise on peut exécuter la requête suivante :

```
SELECT SUM(sales_amount) as total_revenue
FROM sales_data;
```

b) COUNT()

La fonction COUNT() vient ensuite. L'objectif de cette fonction est de compter le nombre de lignes d'un tableau ou le nombre de valeurs non nulles dans une colonne.

Supposons que vous souhaitiez connaître le nombre de produits vendus dans votre magasin ; vous pouvez utiliser la requête suivante :

```
SELECT COUNT(*) as total_products
FROM products;
```

c) AVG()

la fonction AVG() permet de calculer la valeur moyenne d'une colonne numérique. Elle est utile pour rechercher le prix moyen, l'évaluation, les unités vendues, etc. Cette requête, par exemple, serait utile si vous exploitiez un service de diffusion en continu par abonnement et que vous souhaitiez connaître la durée moyenne des sessions des utilisateurs sur votre plateforme :

```
SELECT AVG(session_duration) as average_session_duration
FROM user_sessions;
```

d) MIN()

La fonction `MIN()` renvoie la plus petite valeur d'une colonne. Elle est particulièrement utile pour localiser la valeur la plus faible ou la plus petite d'un ensemble de données.

Prenons l'exemple d'un responsable qui cherche à identifier le temps de réponse le plus rapide à une question posée par un client sur les tickets d'assistance ; la requête suivante peut lui fournir la réponse :

```
SELECT MIN(response_time) as shortest_response_time
FROM customer_support_tickets;
```

e) MAX()

En revanche, la fonction `MAX()` renvoie la valeur la plus élevée d'une colonne. Elle est utile pour déterminer la valeur la plus élevée d'un ensemble de données.

Supposons que vous soyez propriétaire d'un site de commerce électronique et que vous souhaitiez connaître la valeur de la commande la plus élevée passée par un client dans votre boutique en ligne. Voici la requête que vous écrirez :

```
SELECT MAX(order_value) as max_order_value
FROM customer_orders;
```

f) Exploiter la puissance des fonctions agrégées SQL avec GROUP BY

Dans la plupart des cas, l'utilisation des fonctions d'agrégation de SQL implique l'utilisation de la clause GROUP BY. Cette puissante combinaison vous permet d'exécuter des calculs agrégés sur des groupes spécifiques de données dans votre ensemble de données.
La clause SQL GROUP BY sépare votre ensemble de données en groupes sur la base de valeurs correspondantes dans une ou plusieurs colonnes, ce qui vous permet d'appliquer des fonctions d'agrégation à chaque groupe indépendamment. Vous trouverez ci-dessous un aperçu de la syntaxe GROUP BY:

```
SELECT
  column1,
  column2,
  aggregate_function(column3)
FROM table_name
GROUP BY column1, column2;
```

Exemple 1 : regroupement par ville et calcul du salaire moyen
Imaginez que vous disposiez d'une base de données d'employés contenant des informations sur les noms, les villes et les salaires des employés. Votre objectif est de déterminer le salaire moyen dans chaque ville. Vous pouvez le faire avec la requête suivante :

```
SELECT
  city,
  AVG(salary) as average_salary
FROM employee_data
GROUP BY city;
```

Exemple 2 : Regroupement par catégorie de produits et calcul du chiffre d'affaires total
Supposons que vous disposiez d'un ensemble de données de ventes comprenant des catégories de produits et des quantités vendues. Votre objectif est de déterminer les ventes totales pour chaque catégorie de produits. Voici la requête :

```
SELECT
  product_category,
  SUM(sales_amount) as total_sales
FROM sales_data
GROUP BY product_category;
```

Exemple 3 : Regroupement par année et par mois pour compter les commandes
Supposons que vous exploitiez une plateforme de commerce électronique dont la base de données contient les dates de commande. Vous souhaitez connaître le nombre de commandes passées chaque année et chaque mois. La requête suivante permet d'y parvenir :

```
SELECT
  EXTRACT(YEAR FROM order_date) as order_year,
  EXTRACT(MONTH FROM order_date) as order_month,
  COUNT(*) as order_count
FROM orders
GROUP BY order_year, order_month
ORDER BY order_year, order_month;
```

2) Fonctions analytiques

Commençons par un exemple d'utilisation des fonctions analytiques pour calculer une moyenne mobile.

Supposons qu'on ait une table de cotation boursière (les cotations ont lieu les jours ouvrés uniquement) :

```
create table COTATION (
  isin    varchar2 (20),   -- identifiant de la société
  dte     date,            -- date de cotation
  ouv     number,          -- premier cours de la journée
  max     number,          -- cours le plus haut de la journée
  min     number,          -- cours le plus bas de la journée
  clo     number,          -- dernier cours de la journée (clôture)
  volume  number,          -- nb d'actions échangées
  constraint pk_cotation primary key (isin, dte)
)
```

La méthode traditionnelle est :

```
create table MMA_NORMAL as
select
  a.isin,
  a.dte,
  avg(a.clo) mma_5
from
  (
    select   a.isin, a.dte, b.clo
    from
      (
        select   isin, dte, rownum rnum
        from
          (
            select   isin, dte
            from     COTATION
            order by isin, dte
          )
      ) a,
      (
        select   isin, dte, clo, rownum rnum
        from
          (
            select   isin, dte, clo
            from     COTATION
            order by isin, dte
          )
      ) b
    where
      a.isin = b.isin and
      b.rnum between a.rnum-4 and a.rnum
      -- on a besoin d'utiliser les ROWNUM car
      -- les dates ne sont pas forcément consécutives
  ) a
group by isin, dte
```

Avec les fonctions analytiques, la requête s'écrit simplement :

```
create table MMA_ANALYTIC as
select   isin, dte,
        avg(clo) over(partition by isin order by dte rows 4 preceding) mma_5
from     COTATION
```

Explication

avg : c'est la fonction analytique utilisée ici; elle a le même nom que la fonction de groupe AVG mais cette dernière nécessite un group by alors qu'ici on n'en a pas besoin.
over : mot-clé qui indique qu'on utilise une fonction analytique
partition by isin : partition by indique un découpage logique; la fonction analytique utilisée ne calcule qu'à l'intérieur d'une partition. C'est tout à fait similaire au group by dans une requête traditionnelle.
order by dte: à l'intérieur de chaque partition, les lignes sont ordonnées par date.
rows 4 preceding : on considère uniquement la ligne courante et les 4 lignes précédentes.
Donc on a exactement ce que l'on veut : pour une société donnée (partition by isin), on trie les données par dte (order by dte) et pour chaque ligne obtenue et les 4 lignes précédentes (rows 4 preceding) on prend la moyenne des cours de clôture (avg(clo)).

Les fonctions analytiques ou de fenêtre SQL fonctionnent en trois étapes. Tout d'abord, vous définissez une fenêtre. En effet, les fonctions Windows opèrent sur un ensemble de lignes définies à l'aide de la clause **OVER()**. Ensuite, la clause **PARTITION BY** divise l'ensemble des résultats en partitions auxquelles la fonction est appliquée. Enfin, la clause **ORDER** by détermine l'ordre des lignes dans chaque partition.

a) Syntaxe des fonctions SQL de fenêtre

La syntaxe générale des fonctions de fenêtre comprend trois éléments : la fonction elle-même, la clause ***OVER*** et une ***PARTITION BY*** facultative. Voici la décomposition de chacun de ces éléments.

La fonction

La fonction indique l'opération de calcul que vous ciblez sur la ligne. Il peut s'agir de fonctions d'agrégation standard (par exemple, ***SUM***, ***AVG*** et ***COUNT***). Ou d'une fonction analytique (par exemple, ***ROW_NUMBER***, ***RANK***, ***LEAD*** *et* LAG).
Voici un exemple utilisant la fonction ***SUM***.

```
SELECT
   date,
   produit,
   montant,
   SUM(montant) OVER (ORDER BY date) AS cumul_montant
FROM
   ventes;
```

Cette requête calcule le total cumulé des montants de vente pour chaque date. La fonction SUM() est appliquée sur une fenêtre définie par ORDER BY date, c'est-à-dire que pour chaque ligne, le cumul est calculé en additionnant les montants des lignes précédentes, jusqu'à la date courante

La clause ***OVER*** vous permet de définir les lignes sur lesquelles la fonction doit opérer. Il existe deux types de clauses : ***PARTITION BY*** et ***ORDER BY***. La première est facultative et divise les résultats en sections auxquelles la fonction window est appliquée.
Pour le second, il spécifie l'ordre des lignes dans chaque partition. Si elle n'est pas spécifiée, la fonction traite la fenêtre comme un ensemble non ordonné. Voici un exemple.

Clause PARTITION BY

Cette clause, facultative, divise le résultat en partitions auxquelles la fonction window est appliquée. Sa puissance peut être réalisée en effectuant des calculs indépendamment avec chaque partition. En voici une illustration.

```
 SELECT
    departement,
    employe,
    salaire,
    RANK() OVER (PARTITION BY departement ORDER BY salaire DESC) AS
rang_salaire
FROM
    employes;
```

Cette requête calcule le rang de chaque employé par rapport à son salaire dans son département. La clause PARTITION BY departement divise les lignes en groupes (les départements), et la fonction RANK() attribue un rang dans chaque groupe, en ordonnant par salaire décroissant.

b) Fonctions de fenêtre agrégées en SQL

Les fonctions de fenêtre d'agrégat en SQL réutilisent les fonctions d'agrégat existantes, telles que ***COUNT()*** ou ***SUM()***, en modifiant la façon dont l'agrégat est défini et le format des résultats. Cela signifie qu'elles effectuent des calculs sur une fenêtre liée à la ligne courante dans l'ensemble de résultats. Vous pouvez les utiliser pour obtenir des valeurs agrégées basées sur une fenêtre ou un cadre spécifié. Voici un tableau contenant une brève description de chacun d'entre eux.

Fonction	Objectif	Exemple d'utilisation
MIN()	Renvoie la valeur minimale d'une expression spécifiée sur un cadre.	MIN(column) OVER (ORDER BY date_column ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) – Calcule le minimum mobile pour chaque ligne.
MAX()	Renvoie la valeur maximale d'une expression spécifiée sur une fenêtre.	MAX(column) OVER (ORDER BY date_column ROWS UNBOUNDED PRECEDING) – Calcule le maximum mobile sur l'ensemble des résultats
AVG()	AVG() Renvoie la valeur moyenne d'une expression spécifiée sur une trame.	AVG(column) OVER (ORDER BY date_column ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) – Calcule la moyenne mobile sur une fenêtre.
SUM()	Renvoie la somme d'une expression spécifiée sur une fenêtre.	SUM(column) OVER (ORDER BY date_column ROWS UNBOUNDED PRECEDING) – Calcule la somme cumulative sur l'ensemble des résultats.
COUNT()	Renvoie le nombre de lignes dans une fenêtre ou le nombre total de lignes.	COUNT(column) OVER (ORDER BY date_column ROWS UNBOUNDED PRECEDING) – Calcule le nombre de lignes sur l'ensemble des résultats.

c) Fonctions de fenêtre de valeur en SQL

En SQL, les fonctions de fenêtre de valeur sont utilisées pour allouer des valeurs de ligne à partir d'autres lignes. Contrairement aux fonctions d'agrégation, qui renvoient une valeur unique pour chaque groupe, elles renvoient une valeur pour chaque ligne en fonction d'une fenêtre spécifique ou d'un cadre de lignes.
Ce modèle d'opération vous permet d'accéder aux données d'autres lignes par rapport à la fenêtre actuelle, ce qui facilite l'analyse et la création de rapports.
Voici un bref aperçu de chacune des fonctions de fenêtre de valeur.

- **LEAD()**

La fonction de fenêtre SQL ***LEAD()*** fonctionne à l'opposé de ***LAG()*** et est utilisée pour renvoyer les valeurs des lignes suivantes. Pour simplifier, elle décale les valeurs d'une ligne vers le haut. La syntaxe pour l'appeler est similaire à celle de ***LAG()***.
LEAD(column, offset, default_value) OVER (PARTITION BY partition_expression ORDER BY sort_expression)

```
SELECT
   date,
   montant,
   LEAD(montant, 1, 0) OVER (ORDER BY date) AS montant_suivant,
   montant - LEAD(montant, 1, 0) OVER (ORDER BY date) AS difference_suivante
FROM
   operations;
```

Cette requête calcule le montant de l'opération suivante et la différence avec le montant courant. La fonction LEAD() permet d'accéder à la valeur de la ligne suivante (ici, montant avec un décalage de 1 ligne).

Fonction	Objectif	Utilisation Exemple
LEAD()	Renvoie les valeurs des lignes suivantes dans une fenêtre.	**LEAD(column, 1, 0) OVER (ORDER BY date_column) AS lead_value** – Récupère la valeur suivante en fonction du critère d'ordre, avec une valeur par défaut s'il n'y a pas de valeur suivante.
LAG()	Renvoie les valeurs des lignes précédentes dans une fenêtre.	**LAG(column, 1, 0) OVER (ORDER BY date_column) AS lag_value** – Récupère la valeur précédente en fonction du critère d'ordre, avec une valeur par défaut s'il n'y a pas de valeur suivante.
FIRST_VALUE()	Récupère la valeur d'une expression spécifiée pour la première ligne d'une fenêtre.	**FIRST_VALUE(column) OVER (PARTITION BY category_column ORDER BY date_column) AS first_value** – Obtient la première valeur dans chaque catégorie en fonction de l'ordre des dates.
LAST_VALUE()	Récupère la dernière valeur d'une expression spécifiée dans un cadre défini.	**LAST_VALUE(column) OVER (PARTITION BY category_column ORDER BY date_column) AS last_value** – Récupère la dernière valeur de chaque catégorie en fonction de l'ordre des dates.
NTH_VALUE()	Récupère la valeur à une position spécifiée	**NTH_VALUE(column, 3) OVER (PARTITION BY category_column ORDER BY date_column)**

Fonction	Objectif	Utilisation Exemple
	dans le cadre d'une fenêtre.	**AS third_value** – Récupère la troisième valeur de chaque catégorie en fonction de l'ordre des dates.

d) **Fonction de fenêtre de classement en SQL**

Les fonctions de fenêtre de classement en SQL sont utiles pour attribuer des nombres (rangs ou positions) aux lignes d'un ensemble de résultats en fonction d'un critère d'ordre spécifique. Elles sont utiles pour analyser et ordonner les données et identifier les positions relatives des lignes.

- **RANK()**

La fonction ***RANK()*** est la plus courante et attribue des valeurs de classement basées sur l'ordre que vous avez spécifié. Sa syntaxe..

RANK() OVER (PARTITION BY partition_expression ORDER BY sort_expression)

La requête ci-dessous attribue un rang à chaque employé en fonction de ses ventes ; les ventes les plus élevées obtiennent le rang le plus bas.

```
SELECT
  id_employé,
  ventes,
  RANK() OVER (ORDER BY sales DESC) AS sales_rank
FROM  table_des_ventes ;
```

Fonction	Objectif	Exemples d'utilisation
RANK()	Attribue des valeurs de classement en fonction de l'ordre spécifié.	RANK**() OVER (ORDER BY sales DESC) AS sales_rank** – Attribue un rang à chaque employé en fonction de ses ventes.
DENSE_RANK()	Similaire à RANK(), mais traite les égalités sans écarts.	**DENSE_RANK() OVER (ORDER BY test_score DESC) AS score_rank** – Attribue un rang dense aux étudiants en fonction de leurs résultats aux tests.
ROW_NUMBER()	Attribue des numéros de ligne uniques à la suite de la clause ORDER BY.	**ROW_NUMBER() OVER (PARTITION BY category ORDER BY sales DESC) AS row_num** – Attribue des numéros de ligne aux produits en fonction des ventes dans chaque catégorie.
PERCENT_RANK()	Définit le classement relatif sous forme de pourcentage (0 à 1).	**PERCENT_RANK() OVER (ORDER BY revenue DESC) AS revenue_percent_rank** – Attribue un rang en pourcentage aux clients en fonction de leur chiffre d'affaires.
NTILE(nombre_de_seaux)	Divise les lignes en groupes spécifiés	**NTILE(4) OVER (ORDER BY salary) AS salary_quartile** – Divise les employés en quartiles en fonction de leur salaire.
CUME_DIST()	Calcule la distribution cumulative (0 à 1) des valeurs.	**CUME_DIST() OVER (ORDER BY age) AS age_cume_dist** – Évalue la distribution cumulative des clients en fonction de l'âge, représentée sous forme de pourcentage.

V. Gestion des dates, chaînes, conversions et expressions régulières

1) Gestion des dates

a) Comparaisons de dates SQL

La comparaison de dates est importante en SQL car elle permet aux professionnels des données de filtrer et de trier les données au cours de l'analyse. Cette méthode est particulièrement utile pour les rapports financiers ou la gestion des stocks, qui nécessitent une analyse des tendances.

Les opérateurs SQL de comparaison comme >,<,=, <>(!=), >=,<=

```
 SELECT *
FROM Orders
WHERE OrderDate > '2023-01-01';

 -- Select all orders placed on January 1, 2023
SELECT *
FROM Orders
WHERE OrderDate = '2023-01-01';
```

b) Quelques fonctions en PostgreSQL

Chaque SGBD a des fonctions particulières.

Fonction	***Description***	***Exemple***
EXTRACT	La fonction EXTRACT est utilisée pour récupérer des champs de date tels que l'année, le mois ou le jour. Dans la requête suivante, la clause EXTRACT filtre les enregistrements de l'ANNÉE 2023.	SELECT * FROM Orders WHERE EXTRACT(YEAR FROM OrderDate) = 2023;
INTERVAL	La fonction INTERVAL est utilisée dans pour effectuer certains calculs de date. INTERVAL vous permet d'ajouter ou de soustraire des quantités spécifiques de temps à une date.	-- Select all orders placed in the last 30 days SELECT * FROM Orders WHERE OrderDate > NOW() - INTERVAL '30 days';
DATE_PART	La fonction DATE_PART permet d'extraire un sous-champ de la valeur de la date. Cette sous-zone peut contenir la date ou l'heure. La requête suivante renvoie les commandes passées en mars, ce qui donne une valeur de double précision de 3 pour le troisième mois.	-- Select all orders placed in March SELECT * FROM Orders WHERE DATE_PART('month', OrderDate) = 3;
TO_DATE	La fonction TO_DATE convertit une chaîne de caractères donnée en une valeur de date basée sur le format donné. Dans la requête suivante, la valeur de la date est fournie sous forme de chaîne de caractères. Vous convertissez donc la date au format YYYY-MM-DD.	-- Select all orders placed on January 1, 2023, using a string date SELECT * FROM Orders WHERE OrderDate = TO_DATE('2023-01-01', 'YYYY-MM-DD');
BETWEEN	la clause BETWEEN filtre les données dans un intervalle spécifique. Pour les valeurs contenant des dates, la fonction BETWEEN filtre et	-- Select all orders placed between January 1, 2023, and December 31, 2023 SELECT *

	renvoie les lignes entre les dates indiquées. Ces dates sont également incluses dans les résultats.	FROM Orders WHERE OrderDate BETWEEN '2023-01-01' AND '2023-12-31';

2) Chaînes

La gestion des chaînes en SQL se fait principalement grâce aux fonctions de chaîne intégrées, qui permettent de manipuler des données textuelles. Les opérations courantes incluent la concaténation (CONCAT), le changement de casse (LOWER, UPPER), l'extraction (SUBSTRING, LEFT, RIGHT), le remplacement (REPLACE) et le nettoyage (TRIM, LTRIM, RTRIM). Il est également possible de rechercher des sous-chaînes à l'aide de l'opérateur LIKE.

a) **Fonctions de base et de manipulation**

- **CONCAT** : Combine deux chaînes ou plus en une seule.

SELECT CONCAT ('LearnSQL is good', ' and great', ' and fantastic!')
FROM DUAL;

- **LENGTH** ou **LEN** (selon le SGBD) : Retourne la longueur d'une chaîne.
- **REPLACE** : Remplace toutes les occurrences d'une sous-chaîne par une autre.

SELECT REPLACE ('LearnSQL is good!', 'good', 'great!')
***FROM DUAL*;**

- **SUBSTRING** : Extrait une partie d'une chaîne à partir d'une position de départ et sur une longueur spécifiée.

SELECT SUBSTR('LearnSQL',6,3)
FROM DUAL;

- **LEFT** et **RIGHT** : Extrait un certain nombre de caractères depuis le début (LEFT) ou la fin (RIGHT) d'une chaîne.
- **TRIM** : Supprime les espaces au début et à la fin d'une chaîne.

SELECT TRIM(' SQL2017 ')
FROM DUAL;

- **LTRIM** et **RTRIM** : Suppriment respectivement les espaces à gauche et à droite d'une chaîne.

b) Fonctions de mise en forme

- **LOWER** : Convertit une chaîne en minuscules.
- **UPPER** : Convertit une chaîne en majuscules.
- **LPAD** : Ajoute des caractères au début d'une chaîne pour atteindre une longueur désirée (plusieurs SGBD ne gèrent pas ce paramètre de la même manière).

c) Recherche et comparaison

- **LIKE** : Utilise des caractères de remplacement (% pour plusieurs caractères, _ pour un seul) pour rechercher des chaînes qui correspondent à un modèle. La sensibilité à la casse dépend du SGBD.
- **ILIKE** : Similaire à LIKE, mais sans tenir compte de la casse.
- **Opérateurs de comparaison** (=, <>, <, >, <=, >=) : Permettent de comparer des chaînes de caractères et de renvoyer une valeur booléenne

3) Conversions

La conversion des types de données est nécessaire dans de nombreux cas. On peut avoir besoin d'agréger et de comparer des valeurs de différents types pour obtenir des informations significatives.
Le formatage des données pour le reporting et la visualisation est un autre aspect important, car il permet de s'assurer que les données sont présentées dans un format structuré et lisible.
La précision des calculs mathématiques dépend également d'une conversion correcte des types de données. Sans cela, des calculs incorrects peuvent conduire à des résultats trompeurs.
Enfin, pour éviter les erreurs lors de l'insertion ou de la mise à jour des données, il faut manipuler avec soin les types de données. Si les types ne correspondent pas, les opérations peuvent échouer ou produire des résultats inattendus.

a) Conversion implicite

Les moteurs SQL disposent d'un mécanisme intégré pour convertir automatiquement les types de données lorsque cela est nécessaire. Cela signifie que lorsque différents types de données sont utilisés ensemble dans une requête, SQL ajuste souvent le type de données sans nécessiter d'intervention manuelle.
Par exemple, si vous ajoutez un nombre entier à un nombre à virgule flottante, SQL convertira automatiquement le nombre entier en nombre flottant avant d'effectuer l'opération. Cela permet d'assurer la cohérence des calculs et d'éviter les erreurs liées à la non-concordance des types de données.

Exemple :

SELECT 10 + 5.5 AS result; -- SQL convertit automatiquement 10 en 10.0 (float)

Sortie :

result

15.5

Bien que la conversion implicite soit utile et souvent transparente, elle peut parfois conduire à un comportement inattendu, en particulier lorsque l'on travaille avec des chaînes de caractères, des dates ou de grandes valeurs numériques. Comprendre quand et comment SQL effectue ces conversions aide les analystes à écrire des requêtes plus précises et plus efficaces.

b) Conversion explicite

La conversion explicite en SQL consiste à modifier manuellement le type de données d'une valeur pour s'assurer qu'elle est correctement interprétée. Contrairement à la conversion implicite, que SQL gère automatiquement, la conversion explicite nécessite des fonctions SQL spécifiques.
La principale fonction de conversion explicite est CAST. La fonction CAST est normalisée dans les différentes bases de données SQL, ce qui en fait un choix polyvalent.
On a la fonction CONVERT qui est spécifique à SQL Server

Exemple :

SELECT CAST('2024-02-20' AS DATE) AS converted_date;

Sortie :

converted_date

2024-02-20

4) Expressions régulières

Les expressions régulières en SQL sont des motifs de recherche pour des chaînes de caractères, utilisables via des opérateurs comme *REGEXP* (MySQL) ou des fonctions comme *REGEXP_LIKE* (Oracle, SQL Server), permettant des recherches complexes dans les données textuelles. Elles sont particulièrement utiles pour valider des formats (e.g., adresses e-mail, numéros de téléphone), rechercher des motifs spécifiques, et manipuler des textes de manière plus fine que les opérateurs standards comme `LIKE`.

a) Principales utilisations et opérateurs

- **Recherche de motifs**: Définir des modèles pour trouver des correspondances dans des chaînes de caractères.
- **Validation de données**: Vérifier si une chaîne de caractères correspond à un format attendu, comme une adresse e-mail ou un code postal.
- **Opérateurs et fonctions**: La syntaxe peut varier selon le système de base de données. Les plus courants sont :
    - `REGEXP` : Souvent utilisé dans MySQL.
    - `REGEXP_LIKE()` : Utilisé dans des systèmes comme Oracle ou SQL Server.
    - `~` : Utilisé par certains systèmes (comme PostgreSQL) pour la recherche d'expressions régulières.

b) Exemples de syntaxe courante

- **`REGEXP` (MySQL)** :
    - `SELECT * FROM utilisateurs WHERE email REGEXP '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$';`
- **`REGEXP_LIKE()` (Oracle/SQL Server)** :
    - `SELECT * FROM produits WHERE REGEXP_LIKE(nom_produit, '^.{5,15}$');`

c) Caractères et métacaractères utiles

- `^` : Ancre la recherche au début de la chaîne.
- `$` : Ancre la recherche à la fin de la chaîne.
- `.` : Correspond à n'importe quel caractère (sauf les sauts de ligne, sauf optionnellement avec le drapeau `s`).
- `*` : Correspond à zéro ou plusieurs occurrences du caractère ou groupe précédent.
- `+` : Correspond à une ou plusieurs occurrences du caractère ou groupe précédent.
- `[...]` : Correspond à n'importe quel caractère à l'intérieur des crochets, par exemple `[abc]` correspond à 'a', 'b', ou 'c'.
- `[^...]` : Correspond à n'importe quel caractère qui n'est PAS dans les crochets.
- `\\d` : Correspond à n'importe quel chiffre (`0-9`).
- `\\b` : Correspond à une limite de mot.
- `|` : Permet de spécifier des alternatives, par exemple `a|b` correspond à 'a' ou 'b

CHAPITRE 3 : Programmation SQL

Il existe dans les SGBD, des langages procédurales. PL/pgSQL est le langage de procédure de **PostgreSQL,** tout comme le PL/SQL est le langage procédural d'oracle. Ces langages permettent d'ajouter des logiques de programmation comme des boucles et des conditions au langage SQL, les rendant plus puissant pour des tâches complexes. Ils permettent d'écrire des fonctions, des procédures stockées et des déclencheurs, offrant des fonctionnalités de traitement de données et de transactions robustes. Nous allons nous baser sur PL/pgSQL. Ces caractéristiques principales sont :

- **Langage procédural :** Il permet d'écrire des blocs d'instructions structurées qui peuvent inclure des variables, des structures de contrôle (conditions, boucles), des fonctions et des procédures stockées.
- **Traitement des données :** Il est spécialement conçu pour le traitement des données et des requêtes SQL, permettant une manipulation directe des données au sein du code procédural.
- **Fonctions et procédures stockées :** Vous pouvez créer des fonctions et des procédures stockées qui sont sauvegardées dans la base de données et exécutées sur demande, ce qui peut améliorer les performances et l'organisation du code.
- **Déclencheurs (Triggers) :** PL/pgSQL est utilisé pour écrire des déclencheurs qui s'exécutent automatiquement en réponse à des événements sur une table (comme des insertions ou des mises à jour).
- **Similitude avec PL/SQL :** Sa syntaxe est très proche de celle de PL/SQL d'Oracle, ce qui facilite le portage de scripts entre les deux systèmes.
- **Intégration :** Il peut facilement utiliser les types de données, les fonctions et les opérateurs définis dans PostgreSQL, ainsi que d'intégrer des requêtes SQL directement dans les blocs de code.

I. les notions de base

A. Structure de PL/pgSQL

Les fonctions écrites en PL/pgSQL sont définies auprès du serveur en exécutant les commandes CREATE FUNCTION. Une telle commande pourrait ressembler à ceci :

```
CREATE FUNCTION une_fonction(integer, text) RETURNS integer
AS 'texte du corps de la fonction'
LANGUAGE plpgsql;
```

PL/pgSQL est un langage structuré en blocs. Le texte complet du corps d'une fonction doit être un *bloc*. Un bloc est défini comme :

```
[ <<label>> ]
[ DECLARE
   déclarations ]
```

```
BEGIN
   instructions
END [ label ];
```

Chaque déclaration et chaque expression au sein du bloc est terminé par un point-virgule. Un bloc qui apparaît à l'intérieur d'un autre bloc doit avoir un point-virgule après END; néanmoins, le END final qui conclut le corps d'une fonction n'a pas besoin de point-virgule.
Un *label* est seulement nécessaire si vous voulez identifier le bloc à utiliser dans une instruction EXIT ou pour qualifier les noms de variable déclarées dans le bloc. Si un label est écrit après END, il doit correspondre au label donné au début du bloc.
Tous les mots clés sont insensibles à la casse. Les identifiants sont convertis implicitement en minuscule sauf dans le cas de l'utilisation de guillemets doubles. Le comportement est donc identique à celui des commandes SQL habituelles.
Les commentaires fonctionnent de la même manière tant dans du PL/pgSQL que dans le code SQL. Un double tiret (--) commence un commentaire et celui-ci continue jusqu'à la fin de la ligne. Un /* commence un bloc de commentaire qui continue jusqu'au */ correspondant. Les blocs de commentaires peuvent imbriquer les uns dans les autres.
Chaque expression de la section expression d'un bloc peut être un *sous-bloc*. Les sous-blocs peuvent être utilisés pour des groupements logiques ou pour situer des variables locales dans un petit groupe d'instructions. Les variables déclarées dans un sous-bloc masquent toute variable nommée de façon similaire dans les blocs externes pendant toute la durée du sous-bloc. Cependant, vous pouvez accéder aux variables externes si vous qualifiez leur nom du label de leur bloc. Par exemple :

```
CREATE FUNCTION une_fonction() RETURNS integer AS $$
<< blocexterne >>
DECLARE
   quantite integer := 30;
BEGIN
   RAISE NOTICE 'quantité vaut ici %', quantite;  -- affiche 30
   quantite := 50;
   -- Crée un sous-bloc
   DECLARE
      quantite integer := 80;
   BEGIN
      RAISE NOTICE 'quantite vaut ici %', quantite;  -- affiche 80
      RAISE NOTICE 'la quantité externe vaut ici %', blocexterne.quantite;  -- affiche 50
   END;

   RAISE NOTICE 'quantité vaut ici %', quantite;  -- affiche 50

   RETURN quantite;
END;
$$ LANGUAGE plpgsql;
```

B. Affectation et affichage

1) Affectation

L'affectation d'une valeur à une variable PL/pgSQL s'écrit ainsi :
variable { := | = } *expression*;

Exemples :

```
taxe := sous_total * 0.06;
mon_enregistrement.id_utilisateur := 20;
```

2) Utiliser RAISE pour afficher des messages

- **RAISE NOTICE** : Affiche un message informatif. Il utilise des espaces réservés (%) pour insérer des variables.

  Exemple :

```
RAISE NOTICE 'Le nom est : %', variable_nom;
```

- **RAISE WARNING** : Affiche un message d'avertissement.

  Exemple :

```
RAISE WARNING 'Attention : la valeur est vide.';
```

C. Déclaration de variables

Toutes les variables utilisées dans un bloc doivent être déclarées dans la section déclaration du bloc. Les seules exceptions sont que la variable de boucle d'une boucle FOR effectuant une itération sur des valeurs entières est automatiquement déclarée comme variable entière (type integer), et de la même façon une variable de boucle FOR effectuant une itération sur le résultat d'un curseur est automatiquement déclarée comme variable de type record.
Les variables PL/pgSQL peuvent être de n'importe quel type de données tels que integer, varchar et char.

1) Quelques exemples de déclaration de variables

```
id_utilisateur integer;
quantité numeric(5);
url varchar;
ma_ligne nom_table%ROWTYPE;
mon_champ nom_table.nom_colonne%TYPE;
une_ligne RECORD;
```

2) La syntaxe générale d'une déclaration de variable

nom [CONSTANT] *type* [COLLATE *nom_collationnement*] [NOT NULL] [{ DEFAULT | := | = } *expression*];

La clause DEFAULT, si indiquée, spécifie la valeur initiale affectée à la variable quand on entre dans le bloc. Si la clause DEFAULT n'est pas indiquée, la variable est initialisée à la valeur SQL NULL. L'option CONSTANT empêche la modification de la variable après initialisation, de sorte que sa valeur reste constante pour la durée du bloc.. Si NOT NULL est spécifié, l'affectation d'une valeur NULL aboutira à une erreur d'exécution. Les valeurs par défaut de toutes les variables déclarées NOT NULL doivent être précisées, donc non NULL.
Le signe d'égalité (=) peut être utilisé à la place de :=, qui lui est conforme au PL/SQL.
La valeur par défaut d'une variable est évaluée et affectée à la variable à chaque entrée du bloc (pas seulement une fois lors de l'appel de la fonction). Ainsi, par exemple, l'affectation de now() à une variable de type timestamp donnera à la variable l'heure de l'appel de la fonction courante, et non l'heure au moment où la fonction a été précompilée.
Exemples :

```
quantité integer DEFAULT 32;
url varchar := 'http://mysite.com';
id_utilisateur CONSTANT integer := 10;
```

3) types simples

En PL/pgSQL, les "types simples" (ou scalaires) correspondent aux types de données de base pris en charge par PostgreSQL. Ils représentent une valeur unique, par opposition aux types composites (comme les enregistrements ou les tableaux).
Voici les principales catégories de types simples utilisables dans PL/pgSQL :

a) Types Numériques

Ces types stockent des nombres entiers ou décimaux.

- **integer** (int, int4) : Le choix usuel pour les nombres entiers, offrant un bon équilibre entre plage de valeurs, taille de stockage et performance.
- **smallint** (int2) : Pour les petites valeurs entières, utile pour économiser de l'espace disque.
- **bigint** (int8) : Pour les très grands nombres entiers.
- **numeric** (ou decimal) : Pour les nombres à virgule fixe avec une précision et une échelle configurables (par exemple, numeric(10, 2) pour la monnaie).
- **real** et **double precision** : Pour les nombres à virgule flottante (approximatifs).

b) Types Caractères

Ces types gèrent les chaînes de texte.

- **varchar(n)** (ou character varying(n)) : Chaînes de caractères de longueur variable avec une limite maximale spécifiée.
- **char(n)** (ou character(n)) : Chaînes de caractères de longueur fixe.
- **text** : Pour les chaînes de très grande taille, sans limite de taille spécifiée.

c) Types Temporels

Ces types stockent des dates et heures.

- **date** : Stocke la date seule (année, mois, jour).
- **time** : Stocke l'heure seule (avec ou sans fuseau horaire).
- **timestamp** : Stocke la date et l'heure (avec ou sans fuseau horaire).
- **interval** : Stocke une durée ou un intervalle de temps.

d) Type Booléen

- **boolean** (ou bool) : Stocke les valeurs de vérité : TRUE, FALSE, ou NULL.

e) Autres types simples

- **uuid** : Pour stocker des identifiants uniques universels.
- **money** : Pour les valeurs monétaires.
- **bytea** : Pour les données binaires (BLOB).

PL/pgSQL permet d'utiliser tous les types de données supportés par le serveur PostgreSQL

4) ALIAS

nouveaunom ALIAS FOR *anciennom*;

on peut déclarer un alias pour n'importe quelle variable et pas seulement des paramètres de fonction. L'utilisation principale de cette instruction est l'attribution d'un autre nom aux variables aux noms prédéterminés, telles que NEW ou OLD au sein d'une procédure trigger.
Exemples:

```
DECLARE
  anterieur ALIAS FOR old;
  misajour ALIAS FOR new;
```

5) Copie de types

variable%TYPE

%TYPE fournit le type de données d'une variable ou d'une colonne de table. Vous pouvez l'utiliser pour déclarer des variables qui contiendront des valeurs de base de données. Par exemple, disons que vous avez une colonne nommée id_utilisateur dans votre table utilisateurs. Pour déclarer une variable du même type de données que utilisateurs.id_utilisateur, vous pouvez écrire :
id_utilisateur utilisateurs.id_utilisateur%TYPE;

6) Types ligne

nom nom_table%ROWTYPE;
nom nom_type_composite;

Une variable de type composite est appelée variable *ligne* (ou variable *row-type*). Une telle variable peut contenir une ligne entière de résultat de requête **SELECT** ou **FOR**, du moment que l'ensemble de colonnes de la requête correspond au type déclaré de la variable. Les champs individuels de la valeur row sont accessibles en utilisant la notation pointée, par exemple varligne.champ.
Une variable ligne peut être déclarée de façon à avoir le même type que les lignes d'une table ou d'une vue existante, en utilisant la notation *nom_table*%ROWTYPE. Elle peut aussi être déclarée en donnant un nom de type composite. Chaque table ayant un type de données associé du même nom, il importe peu dans PostgreSQL que vous écriviez %ROWTYPE ou pas. Cependant, la forme utilisant %ROWTYPE est plus portable.
Voici un exemple d'utilisation des types composites. table1 et table2 sont des tables:

```
CREATE FUNCTION assemble_champs(t_ligne table1) RETURNS text AS $$
DECLARE
   t2_ligne table2%ROWTYPE;
BEGIN
   SELECT * INTO t2_ligne FROM table2 WHERE ... ;
   RETURN t_ligne.f1 || t2_ligne.f3 || t_ligne.f5 || t2_ligne.f7;
END;
$$ LANGUAGE plpgsql;

SELECT assemble_champs(t.*) FROM table1 t WHERE ... ;
```

7) Types record

nom RECORD;

Les variables record sont similaires aux variables de type ligne mais n'ont pas de structure prédéfinie. Elles empruntent la structure effective de type ligne de la ligne à laquelle elles sont affectées durant une commande **SELECT** ou **FOR**. La sous-structure d'une variable record peut changer à chaque fois qu'on l'affecte. Une conséquence de cela est qu'elle n'a pas de sous-structure jusqu'à ce qu'elle ait été affectée, et toutes les tentatives pour accéder à un de ses champs entraînent une erreur d'exécution.

D. Structures de contrôle

Les structures de contrôle sont probablement la partie la plus utile (et importante) de PL/pgSQL. Grâce aux structures de contrôle de PL/pgSQL, vous pouvez manipuler les données PostgreSQL de façon très flexible et puissante.

1) Retour d'une fonction

Il y a deux commandes disponibles qui vous permettent de renvoyer des données d'une fonction : **RETURN** et **RETURN NEXT**.

2) Contrôles conditionnels

a) L'instruction IF

- ***IF-THEN***

```
IF v_id_utilisateur <> 0 THEN
    UPDATE utilisateurs SET email = v_email WHERE id_utilisateur =
v_id_utilisateur;
END IF;
```

- ***IF-THEN-ELSE***

```
IF id_parent IS NULL OR id_parent = ''
THEN
    RETURN nom_complet;
ELSE
    RETURN hp_true_filename(id_parent) || '/' || nom_complet;
END IF;

IF v_nombre > 0 THEN
    INSERT INTO nombre_utilisateurs (nombre) VALUES (v_nombre);
    RETURN 't';
ELSE
    RETURN 'f';
END IF;
```

- ***IF-THEN-ELSIF***

```
IF nombre = 0 THEN
    resultat := 'zero';
ELSIF nombre > 0 THEN
    resultat := 'positif';
ELSIF nombre < 0 THEN
    resultat := 'negatif';
ELSE
    -- hmm, la seule possibilité est que le nombre soit NULL
    resultat := 'NULL';
END IF;
```

Le mot clé ELSIF peut aussi s'écrire ELSEIF.

b) L'instruction CASE

- ***CASE simple***

La forme simple de **CASE** fournit une exécution conditionnelle basée sur l'égalité des opérandes. L'*expression-recherche* est évaluée (une fois) puis comparée successivement à chaque *expression* dans les clauses WHEN. Si une correspondance est trouvée, alors les *instructions* correspondantes sont exécutées, puis le contrôle est passé à la prochaine instruction après END CASE. (Les autres expressions WHEN ne sont pas testées.) Si aucune

correspondance n'est trouvée, les *instructions* du bloc ELSE sont exécutées ; s'il n'y a pas de bloc ELSE, une exception CASE_NOT_FOUND est levée.
Voici un exemple simple :

```
CASE x
   WHEN 1, 2 THEN
      msg := 'un ou deux';
   ELSE
      msg := 'autre valeur que un ou deux';
END CASE;
```

- ***CASE recherché***

La forme recherchée de **CASE** fournit une exécution conditionnelle basée sur la vérification d'expressions booléennes. Chaque *expression-booléenne* de la clause WHEN est évaluée à son tour jusqu'à en trouver une qui est validée (true). Les *instructions* correspondantes sont exécutées, puis le contrôle est passé à la prochaine instruction après END CASE. (Les expressions WHEN suivantes ne sont pas testées.) Si aucun résultat vrai n'est trouvé, les *instructions* du bloc ELSE sont exécutées. Si aucun bloc ELSE n'est présent, une exception CASE_NOT_FOUND est levée.
Voici un exemple :

```
CASE
   WHEN x BETWEEN 0 AND 10 THEN
      msg := 'valeur entre zéro et dix';
   WHEN x BETWEEN 11 AND 20 THEN
      msg := 'valeur entre onze et vingt';
END CASE;
```

3) Boucles simples

Grâce aux instructions LOOP, EXIT, CONTINUE, WHILE FOR et FOREACH, vous pouvez faire en sorte que vos fonctions PL/pgSQL répètent une série de commandes.

a) LOOP

```
[<<label>>]
LOOP
   instructions
END LOOP [ label ];
```

LOOP définit une boucle inconditionnelle répétée indéfiniment jusqu'à ce qu'elle soit terminée par une instruction EXIT ou **RETURN**. Le *label* optionnel peut être utilisé par les instructions EXIT et CONTINUE dans le cas de boucles imbriquées pour définir la boucle impliquée.

b) EXIT

```
EXIT [ label ] [ WHEN expression-booléenne ];
```

Si aucun *label* n'est donné, la boucle la plus imbriquée se termine et l'instruction suivant END LOOP est exécutée. Si un *label* est donné, ce doit être le label de la boucle, du bloc courant ou d'un niveau moins imbriqué. La boucle ou le bloc nommé se termine alors et le contrôle continue avec l'instruction située après le END de la boucle ou du bloc correspondant.
Si WHEN est spécifié, la sortie de boucle ne s'effectue que si *expression-booléenne* est vraie. Sinon, le contrôle passe à l'instruction suivant le EXIT.
EXIT peut être utilisé pour tous les types de boucles ; il n'est pas limité aux boucles non conditionnelles.

```
LOOP
```

```
   -- quelques traitements
   IF nombre > 0 THEN
      EXIT;  -- sortie de boucle
   END IF;
END LOOP;

LOOP
   -- quelques traitements
   EXIT WHEN nombre > 0;
END LOOP;

<<un_bloc>>
BEGIN
   -- quelques traitements
   IF stocks > 100000 THEN
      EXIT un_bloc;  -- cause la sortie (EXIT) du bloc BEGIN
   END IF;
   -- les traitements ici seront ignorés quand stocks > 100000
END;
```

c) CONTINUE

CONTINUE [*label*] [WHEN *expression-booléenne*];

CONTINUE peut être utilisé avec tous les types de boucles ; il n'est pas limité à l'utilisation des boucles inconditionnelles.

Exemples :

```
LOOP
   -- quelques traitements
   EXIT WHEN nombre > 100;
   CONTINUE WHEN nombre < 50;
   -- quelques traitements pour nombre IN [50 .. 100]
END LOOP;
```

d) WHILE

[<<*label*>>]
WHILE *expression-booléenne* LOOP
   instructions
END LOOP [*label*];

L'instruction WHILE répète une séquence d'instructions aussi longtemps que *expression-booléenne* est évaluée à vrai. L'expression est vérifiée juste avant chaque entrée dans le corps de la boucle.

Par exemple :

```
WHILE montant_possede > 0 AND balance_cadeau > 0 LOOP
   -- quelques traitements ici
END LOOP;

WHILE NOT termine LOOP
   -- quelques traitements ici
END LOOP;
```

e) FOR (variante avec entier)

[<<*label*>>]
FOR *nom* IN [REVERSE] *expression* .. *expression* [BY *expression*] LOOP
instruction
END LOOP [*label*];

Cette forme de FOR crée une boucle qui effectue une itération sur une plage de valeurs entières. La variable *nom* est automatiquement définie comme un type integer et n'existe que dans la boucle (toute définition de la variable est ignorée à l'intérieur de la boucle).

```
FOR i IN 1..10 LOOP
   -- prend les valeurs 1,2,3,4,5,6,7,8,9,10 dans la boucle
END LOOP;
FOR i IN REVERSE 10..1 LOOP
   -- prend les valeurs 10,9,8,7,6,5,4,3,2,1 dans la boucle
END LOOP;
FOR i IN REVERSE 10..1 BY 2 LOOP
   -- prend les valeurs 10,8,6,4,2 dans la boucle
END LOOP;
```

4) Boucler dans les résultats de requêtes

En utilisant un type de FOR différent, vous pouvez itérer au travers des résultats d'une requête et par là-même manipuler ces données. La syntaxe est la suivante :

[<<*label*>>]
FOR *cible* IN *requête* LOOP
instructions
END LOOP [*label*];

La *cible* est une variable de type record, row ou une liste de variables scalaires séparées par une virgule. La *cible* est affectée successivement à chaque ligne résultant de la *requête* et le corps de la boucle est exécuté pour chaque ligne. Voici un exemple :

```
CREATE FUNCTION rafraichir_vuemat() RETURNS integer AS $$
DECLARE
   vues_mat RECORD;
BEGIN
   RAISE NOTICE 'Rafraichissement des vues matérialisées...';

   FOR vues_mat IN
     SELECT n.nspname AS mv_schema,
          c.relname AS mv_name,
          pg_catalog.pg_get_userbyid(c.relowner) AS owner
       FROM pg_catalog.pg_class c
   LEFT JOIN pg_catalog.pg_namespace n ON (n.oid = c.relnamespace)
      WHERE c.relkind = 'm'
    ORDER BY 1
   LOOP

    -- Now "mviews" has one record with information about the materialized view

    RAISE NOTICE 'Refreshing materialized view %.% (owner: %)...',
              quote_ident(mviews.mv_schema),
```

```
            quote_ident(mviews.mv_name),
            quote_ident(mviews.owner);
    EXECUTE format('REFRESH MATERIALIZED VIEW %I.%I', mviews.mv_schema,
mviews.mv_name);

  END LOOP;

  RAISE NOTICE 'Fin du rafraichissement des vues matérialisées.';
  RETURN 1;
END;
$$ LANGUAGE plpgsql;
```

La *requête* utilisée dans ce type d'instruction FOR peut être toute commande SQL qui renvoie des lignes à l'appelant : **SELECT** est le cas le plus commun mais vous pouvez aussi utiliser **INSERT**, **UPDATE** ou **DELETE** avec une clause RETURNING. Certaines commandes comme **EXPLAIN** fonctionnent aussi.
L'instruction FOR-IN-EXECUTE est un moyen d'itérer sur des lignes :

```
[<<label>>]
FOR target IN EXECUTE text_expression [ USING expression [, ...] ] LOOP
   instructions
END LOOP [ label ];
```

5) Boucler dans des tableaux

La boucle FOREACH ressemble beaucoup à une boucle FOR mais, au lieu d'itérer sur les lignes renvoyées par une requêtes SQL, elle itère sur les éléments d'une valeur de type tableau. L'instruction FOREACH pour boucler sur un tableau est :

```
[ <<label>> ]
FOREACH target [ SLICE nombre ] IN ARRAY expression LOOP
   instructions
END LOOP [ label ];
```

Sans SLICE ou si SLICE 0 est indiqué, la boucle itère au niveau des éléments individuels du tableau produit par l'évaluation de l'*expression*. La variable *cible* se voit affectée chaque valeur d'élément en séquence, et le corps de la boucle est exécuté pour chaque élément. Voici un exemple de boucle sur les éléments d'un tableau d'entiers :

```
CREATE FUNCTION somme(int[]) RETURNS int8 AS $$
DECLARE
  s int8 := 0;
  x int;
BEGIN
  FOREACH x IN ARRAY $1
  LOOP
    s := s + x;
  END LOOP;
  RETURN s;
END;
$$ LANGUAGE plpgsql;
```

Les éléments sont parcourus dans l'ordre de leur stockage, quelque soit le nombre de dimensions du tableau. Bien que la *cible* est habituellement une simple variable, elle peut être

une liste de variables lors d'une boucle dans un tableau de valeurs composites (des enregistrements). Dans ce cas, pour chaque élément du tableau, les variables se voient affectées les colonnes de la valeur composite.
Avec une valeur SLICE positive, FOREACH itère au travers des morceaux du tableau plutôt que des éléments seuls. La valeur de SLICE doit être un entier constant, moins large que le nombre de dimensions du tableau. La variable *cible* doit être un tableau et elle reçoit les morceaux successifs de la valeur du tableau, où chaque morceau est le nombre de dimensions indiquées par SLICE. Voici un exemple d'itération sur des morceaux à une dimension :

```
CREATE FUNCTION parcourt_lignes(int[]) RETURNS void AS $$
DECLARE
  x int[];
BEGIN
  FOREACH x SLICE 1 IN ARRAY $1
  LOOP
    RAISE NOTICE 'ligne = %', x;
  END LOOP;
END;
$$ LANGUAGE plpgsql;

SELECT parcourt_lignes(ARRAY[[1,2,3],[4,5,6],[7,8,9],[10,11,12]]);

NOTICE:  ligne = {1,2,3}
NOTICE:  ligne = {4,5,6}
NOTICE:  ligne = {7,8,9}
NOTICE:  ligne = {10,11,12}
```

6) Récupérer les erreurs

Par défaut, toute erreur survenant dans une fonction PL/pgSQL annule l'exécution de la fonction mais aussi de la transaction qui l'entoure. Vous pouvez récupérer les erreurs en utilisant un bloc **BEGIN** avec une clause EXCEPTION. La syntaxe est une extension de la syntaxe habituelle pour un bloc **BEGIN** :

```
 [ <<label>> ]
 [ DECLARE
   declarations ]
 BEGIN
 instructions
 EXCEPTION
 WHEN condition [ OR condition ... ] THEN
 instructions_gestion_erreurs
 [ WHEN condition [ OR condition ... ] THEN
   instructions_gestion_erreurs
   ... ]
 END;
```

```
WHEN division_by_zero THEN ...
  WHEN SQLSTATE '22012' THEN ...
```

Si une nouvelle erreur survient à l'intérieur des *instructions_gestion_erreurs* sélectionnées, elle ne peut pas être récupérée par cette clause EXCEPTION mais est propagée en dehors. Une clause EXCEPTION l'englobant pourrait la récupérer.

Quand une erreur est récupérée par une clause EXCEPTION, les variables locales de la fonction PL/pgSQL restent dans le même état qu'au moment où l'erreur est survenue mais toutes les modifications à l'état persistant de la base de données à l'intérieur du bloc sont annulées. Comme exemple, considérez ce fragment :

```
INSERT INTO mon_tableau(prenom, nom) VALUES('Tom', 'Jones');
BEGIN
  UPDATE mon_tableau SET prenom = 'Joe' WHERE nom = 'Jones';
  x := x + 1;
  y := x / 0;
  EXCEPTION
    WHEN division_by_zero THEN
      RAISE NOTICE 'récupération de l''erreur division_by_zero';
RETURN x;
END;
```

II. Création de procédures et fonctions stockées

Une procédure ou fonction stockée est un ensemble d'instructions SQL pré-compilées, stockées dans la base de données, qui peut être exécuté à la demande. Contrairement à une fonction, une procédure n'est pas conçue pour retourner un résultat et peut gérer les transactions (commit/rollback).

A. Les fonctions

1) Définition d'une fonction personnalisée

Une fonction permet d'encapsuler des calculs ou des opérations souvent répétées dans la base de données et les réutiliser dans des requêtes SQL. Cela simplifie la maintenance et permet une centralisation de la logique métier.
Voici quelques cas d'usages d'une fonction :

- **Réutilisation** : une fonction peut être appelée dans plusieurs requêtes, évitant de répéter le même code ;
- **Facilité de maintenance** : si une modification est nécessaire, elle est effectuée au niveau de la fonction et sera automatiquement appliquée à toutes les requêtes qui l'utilisent ;
- **Centralisation** : Plutôt que de disperser la logique dans plusieurs requêtes SQL, vous pouvez la regrouper au sein d'une fonction, assurant ainsi une plus grande cohérence dans l'application des règles métier.

Une fonction est définie avec un ou plusieurs paramètres, permettant de passer des valeurs en entrée pour traiter des calculs ou manipulations spécifiques. En PostgreSQL, une fonction peut retourner un seul résultat ou une table entière, selon sa définition.
La syntaxe d'une fonction est la suivante :

```
CREATE [or REPLACE] FUNCTION function_name(param_list)
RETURNS return_type
LANGUAGE plpgsql
as
$$
DECLARE
-- déclaration des variables
BEGIN
-- logique de code
END;
```

```
$$;
```

- CREATE [or REPLACE] FUNCTION function_name(param_list) : Cette commande crée une nouvelle fonction avec le nomfunction_name. L'option or replace permet de remplacer une fonction existante avec le même nom.
- param_list : La liste des paramètres que la fonction prend en entrée. Chaque paramètre est défini avec un nom et un type de données.
- RETURNS return_type: Définit le type de données que la fonction retourne. Cela peut être un type simple (comme integer, text) ou composite (comme table).
- LANGUAGE plpgsql: Indique que la fonction est écrite en PL/pgSQL, le langage procédural de PostgreSQL.
- DECLARE: Section optionnelle où l'on peut déclarer des variables locales à utiliser dans la fonction.
- BEGIN ... END: Contient la logique de la fonction, où vous pouvez écrire des instructions SQL et PL/pgSQL (boucles, conditions, etc.). C'est ici que le traitement est effectué.
- $$: Délimiteur pour définir le bloc de code PL/pgSQL.

Exemple de la fonction pour calculer les frais de retard.

```
CREATE FUNCTION calculate_late_fee(rental_date TIMESTAMP, return_date
TIMESTAMP)
RETURNS NUMERIC AS $$
DECLARE
late_fee NUMERIC;
days_late INTEGER;
BEGIN
IF return_date > rental_date + INTERVAL '3 days' THEN
days_late := EXTRACT(DAY FROM (return_date - rental_date - INTERVAL '3 days'));
late_fee := days_late * 1.5; -- 1,5 € par jour de retard
ELSE
late_fee := 0;
END IF;
RETURN late_fee;
END;
$$ LANGUAGE plpgsql;
```

- La fonction prend en paramètres la date de location (rental_date) et la date de retour réelle (return_date).
- Si le film est rendu plus de 3 jours après la date de location, elle calcule le nombre de jours de retard et applique une pénalité de 1,5 € par jour.
- Sinon, aucuns frais de retard ne sont appliqués.

Définissons maintenant celle associée à l'anonymisation des emails :

```
CREATE FUNCTION anonymize_email(email VARCHAR)
RETURNS VARCHAR AS $$
BEGIN
RETURN SUBSTRING(email FROM 1 FOR 3) || '***';
END;
$$ LANGUAGE plpgsql;
```

- La fonction prend en paramètre une adresse email.
- Elle retourne une version anonymisée, ne montrant que les trois premiers caractères suivis de ***.

2) *Appel d'une fonction*

On peut appeler les fonctions directement dans la clause SELECT ou dans un autre bloc. Examinons d'abord le résultat de la fonction calculate_late_feeen utilisant les transactions de notre table rental :

```
SELECT
r.rental_id,
c.first_name,
c.last_name,
calculate_late_fee(r.rental_date, r.return_date) AS late_fee
FROM
rental r
JOIN
customer c ON r.customer_id = c.customer_id
WHERE
r.return_date IS NOT NULL
ORDER BY
rental_id;
```

Data Output | Messages | Notifications

SQL

	rental_id integer	first_name character varying (45)	last_name character varying (45)	late_fee numeric
1	1	Charlotte	Hunter	0
2	2	Tommy	Collazo	0.0
3	3	Manuel	Murrell	6.0
4	4	Andrew	Purdy	9.0
5	5	Delores	Hansen	7.5
6	6	Nelson	Christenson	0
7	7	Cassandra	Walters	1.5
8	8	Minnie	Romero	0.0
9	9	Ellen	Simpson	0.0
10	10	Danny	Isom	4.5

Total rows: 1000 of 15861 Query complete 00:00:00.064 Ln 12, Col 1

Capture d'écran d'un tableau de résultats de pgAdmin avec quatre colonnes

Examinons ensuite le résultat de la fonction anonymize_email en retournant des informations associées à nos clients :

```
SELECT
c.customer_id,
c.first_name,
c.last_name,
anonymize_email(c.email) AS email_anonymized
FROM
customer c
ORDER BY
customer_id;
```

Data Output | Messages | Notifications

SQL

	customer_id [PK] integer	first_name character varying (45)	last_name character varying (45)	email_anonymized character varying
1	1	Mary	Smith	mar***
2	2	Patricia	Johnson	pat***
3	3	Linda	Williams	lin***
4	4	Barbara	Jones	bar***
5	5	Elizabeth	Brown	eli***
6	6	Jennifer	Davis	jen***
7	7	Maria	Miller	mar***
8	8	Susan	Wilson	sus***
9	9	Margaret	Moore	mar***
10	10	Dorothy	Taylor	dor***

Total rows: 599 of 599 Query complete 00:00:00.052 Ln 8, Col 1

B. Les procédures

Les procédures permettent de factoriser le code, de respecter des règles métier, d'améliorer les performances et d'automatiser des tâches complexes, et s'exécutent en utilisant la commande CALL.

Les procédures stockées sont des blocs de code SQL qui peuvent encapsuler plusieurs opérations, comme des mises à jour, insertions, suppressions ou vérifications complexes. Contrairement aux fonctions, qui retournent généralement une valeur, les procédures stockées peuvent ne retourne pas de valeur, mais plutôt exécuter des actions complexes sur les données. Pourquoi utiliser des procédures stockées ?

- **Automatisation** : elles permettent d'automatiser des tâches répétitives sans intervention manuelle ;
- **Centralisation de la logique métier** : les règles et traitements complexes sont centralisés au sein de la procédure, facilitant la maintenance.
- **Sécurité et performance** : en encapsulant des processus dans une procédure stockée, vous limitez les risques d'erreurs humaines et optimisez les performances car les procédures stockées sont précompilées.

1) Création d'une procédure stockée

La création d'une procédure stockée suit une syntaxe similaire à celle d'une fonction, mais sans le besoin de retourner une valeur. Voici la syntaxe de base:

```
CREATE PROCEDURE procedure_name(param_list)
LANGUAGE plpgsql
AS $$
BEGIN
-- logique de la procédure
END;
$$;
```

Explication :

- CREATE PROCEDURE procedure_name(param_list): Crée une nouvelle procédure nommée procedure_name avec une liste de paramètres (optionnelle). CREATE OR

REPLACE PROCEDURE va définir une nouvelle procédure, ou remplacer une définition existante.

- LANGUAGE plpgsql : Indique que la procédure est écrite en PL/pgSQL, le langage procédural de PostgreSQL.
- BEGIN ... END: C'est ici que la logique de la procédure est définie. Vous pouvez y inclure plusieurs instructions SQL.

Exemple nous allons créer une procédure stockée qui archive automatiquement les locations retournées depuis plus d'un an dans une table dédiée (rental_archive), puis supprime ces enregistrements de la table des locations actives (rental) :

```
CREATE PROCEDURE archive_old_rentals()
LANGUAGE plpgsql
AS $$
BEGIN
-- Archiver les locations dans une table d'archive
INSERT INTO rental_archive (rental_id, customer_id, rental_date, return_date)
SELECT rental_id, customer_id, rental_date, return_date
FROM rental
WHERE return_date < NOW() - INTERVAL '1 year';
-- Mettre à jour la table payment pour assigner une valeur au lieu de NULL
UPDATE payment
SET rental_id = -1 -- ou une autre valeur spéciale
WHERE
rental_id IN (
SELECT rental_id
FROM rental
WHERE return_date < NOW() - INTERVAL '1 year'
);
-- Supprimer les locations archivées de la table rental
DELETE FROM rental
WHERE return_date < NOW() - INTERVAL '1 year';
END;
$$;
```

Explication :

- INSERT INTO rental_archive: Cette partie de la procédure stocke dans une table d'archive toutes les locations terminées depuis plus d'un an.
- DELETE FROM rental: Après avoir archivé les données, on supprime les enregistrements correspondants de la table des locations actives (rental).

Avant de pouvoir appeler la procédure archive_old_rentals, vous devez d'abord créer la table rental_archive dans votre base de données. Voici le schéma de la table à créer :

```
CREATE TABLE rental_archive (
rental_id INTEGER PRIMARY KEY,
customer_id INTEGER NOT NULL,
rental_date TIMESTAMP NOT NULL,
return_date TIMESTAMP,
insertion_date TIMESTAMP DEFAULT now()
);
```

2) Exécution de la procédure stockée

Pour exécuter une procédure stockée, il suffit d'utiliser la commande CALL:

```
CALL archive_old_rentals();
```

III. Création et utilisation des triggers

1) Présentation

Un trigger dans PostgreSQL est une fonction qui est automatiquement déclenchée lorsqu'un événement spécifique survient sur une table. Ces événements peuvent inclure :INSERT, UPDATE, DELETE ou TRUNCATE.
En utilisant des triggers, vous pouvez automatiser certaines actions lorsque ces événements se produisent, sans intervention manuelle.
PostgreSQL offre deux principaux types de triggers :

- Row-level triggers : Le trigger est déclenché pour chaque ligne affectée par l'événement.
- Statement-level triggers : Le trigger est déclenché une seule fois, quel que soit le nombre de lignes affectées.

La principale différence entre ces deux types réside dans la fréquence et le moment où ils sont déclenchés : si vous exécutez une instruction UPDATE qui modifie 20 lignes, un row-level trigger sera invoqué 20 fois (une fois pour chaque ligne modifiée), tandis qu'un statement-level trigger sera invoqué une seule fois, au moment où l'instruction UPDATE est exécutée.

2) Mise en place d'un trigger

Pour automatiser encore davantage l'archivage des locations terminées, vous pouvez associer un row-level trigger à la table rental. Ce trigger appellera automatiquement la procédure archive_old_rentals chaque fois qu'une modification est faite sur la date de retour d'une location. Si la location a plus d'un an, elle sera archivée automatiquement.
Voici comment vous pouvez créer ce trigger :

```
CREATE TRIGGER archive_trigger_rental
AFTER UPDATE ON rental
FOR EACH ROW
WHEN (NEW.return_date IS NOT NULL AND NEW.return_date < NOW() - INTERVAL '1 year')
EXECUTE PROCEDURE archive_old_rentals();
```

Explication :

- AFTER UPDATE ON rental: Le trigger sera déclenché après chaque mise à jour sur la table rental.
- FOR EACH ROW: C'est un row-level trigger, donc il sera déclenché pour chaque ligne modifiée.
- WHEN (NEW.return_date IS NOT NULL AND NEW.return_date < NOW() - INTERVAL '1 year') : La condition vérifie que la nouvelle date de retour (NEW.return_date) n'est pas nulle et que la location date de plus d'un an.
- EXECUTE PROCEDURE archive_old_rentals(): Si la condition est remplie, la procédure archive_old_rentals est exécutée, archivant automatiquement les locations concernées.

Les triggers sont un excellent moyen d'automatiser des processus critiques, tels que l'archivage ou la validation de données, sans nécessiter d'intervention manuelle.

IV. Transactions & contrôle de concurrence (COMMIT, ROLLBACK, SAVEPOINT)

les problèmes suivants se posent souvent dans les bases de données :

- De multiples utilisateurs doivent pouvoir accéder à la base de données en même temps donc un problème d'accès concurrents ;

- De nombreuses et diverses pannes peuvent apparaître. Il ne faut pourtant pas perdre les données.

La Gestion de transactions répond à ces problèmes.

1) Définition

- Les transactions SQL sont des séquences d'opérations traitées comme une unité unique, soit toutes réussies, soit toutes annulées, garantissant l'intégrité des données. Une transaction est une suite d'opérations interrogeant et/ou modifiant la BD, pour laquelle l'ensemble des opérations doit être soit validé, soit annulé. Toute la transaction est réalisée ou rien ne l'est. Une transaction est un ensemble d'instructions (comme INSERT, UPDATE, DELETE) traitées comme une seule unité de travail. Soit toutes les opérations réussissent, soit aucune ne l'est.

– Validation : toute la transaction est prise en compte ;
– Avortement ou annulation : la transaction n'a aucun effet.

- Le contrôle de concurrence gère l'accès simultané de plusieurs utilisateurs pour éviter les problèmes tels que les lectures erronées. Les commandes clés sont BEGIN TRANSACTION (ou BEGIN), COMMIT (pour valider) et ROLLBACK (pour annuler).

2) ACID

Les transactions respectent les propriétés ACID :

- **Atomicité**

Soit toutes les modifications effectuées par une transaction sont enregistrées dans la BD, soit aucune ne l'est.

- **Cohérence**

Une transaction fait passer une BD d'un état cohérent à un autre état cohérent. Un état cohérent est un état dans lequel les contraintes d'intégrité sont vérifiées.

- **Isolation**

Une transaction se déroule sans être perturbée par les transactions concurrentes : tout se passe comme si elle se déroulait seule.

- **Durabilité**

Une fois qu'une transaction a été confirmée le SGBD garantit qu'aucune modification qu'elle a effectuée ne sera perdue même en cas : interruption, pannes du système d'exploitation, « crash » de disque, etc

3) Commandes

- BEGIN TRANSACTION : Démarre une nouvelle transaction.
- COMMIT : Valide toutes les modifications faites dans la transaction, les rendant permanentes.
- ROLLBACK : Annule toutes les modifications depuis le début de la transaction.
- **Mode de validation automatique** : Si activé, chaque commande est validée immédiatement sans qu'un COMMIT ou ROLLBACK ne soit nécessaire.

4) Contrôle de concurrence

- **Objectif** : Prévenir les problèmes lorsque plusieurs utilisateurs accèdent et modifient les données simultanément.
- **Mécanismes** :
    - **Verrouillage** : Les transactions posent des verrous sur les données qu'elles utilisent pour empêcher d'autres transactions d'accéder à ces données de manière conflictuelle.

    - **Verrouillage Pessimiste** : Prévention des conflits en utilisant des verrous avant d'effectuer les opérations.
    - **Verrouillage Optimiste** : Détection et résolution des conflits après qu'ils se soient produits.
- **Exemple de problème** : Un transfert d'argent où le retrait d'un compte est validé mais l'ajout au compte de destination échoue. Une transaction assure que ces deux étapes sont traitées comme une seule.

V. Gestion des droits et sécurité des accès (GRANT, REVOKE)

1) Création d'un utilisateur

- Syntaxe PostgreSQL

CREATE USER '<user>'

- Syntaxe Oracle

CREATE <user> IDENTIFIED BY <Mot de passe>;

- Syntaxe MySQL

CREATE '<user>' IDENTIFIED BY '<Mot de passe>';

2) Changer le mot de passe d'un utilisateur

- Syntaxe Oracle

ALTER USER <user> IDENTIFIED BY <Nouveau Mot de passe>;

- Syntaxe MySQL

SET PASSWORD FOR '<user>'@'<host>' = PASSWORD('mypass');

- Syntaxe PostgreSQL

ALTER USER <user> PASSWORD 'mypass';

3) Attribuer des droits (GRANT)

À partir de ces quelques tables d'exemples, nous allons pouvoir attribuer les droits aux utilisateurs avec les ordres GRANT et REVOKE.
L'attribution des droits est réservée aux créateurs de la table, néanmoins l'administrateur de la base peut donner l'autorisation à un autre user la possibilité de gérer les droits sur les tables.
La commande GRANT permet d'attribuer par utilisateur de la base de données des accès sur une ou plusieurs tables.
Les droits les plus couramment utilisés sont :

- SELECT : autorise la sélection de données.
- UPDATE : autorise la modification de données.
- DELETE : autorise la suppression de données.
- INSERT : autorise l'insertion de données.

La syntaxe est la suivante
GRANT <droit1>, <droit2>, ...
ON TABLE <nom table>
TO <user1>, <user2> ...
[WITH GRANT OPTION]
Pour attribuer tous les droits, il faut utiliser la syntaxe suivante :

- Syntaxe Oracle et PostgreSQL

GRANT ALL PRIVILEGES
ON <table1>, <table2>, ...
[WITH GRANT OPTION] TO <user1>, <user2> ...

- Syntaxe MySQL

GRANT ALL PRIVILEGES
ON TABLE <table1>, <table2>, ...
[WITH GRANT OPTION] TO <user1>, <user2> ..

La clause WITH GRANT OPTION donne l'autorisation aux users désignés d'attribuer des droits sur ces tables à d'autres utilisateurs.

4) Supprimer des droits (REVOKE)

Une fois les droits attribués, il faut pouvoir les enlever si l'utilisateur quitte l'entreprise par exemple ou change de service. Pour ceci, nous utiliserons l'ordre REVOKE. Ne pas confondre DENY, qui interdit, avec REVOKE qui supprime un droit, sans interdire un accès.

- La syntaxe Oracle est la suivante

REVOKE <droit1>, <droit2>, ...
ON TABLE <nom table>
FROM <user1>, <user2> ...;

- La syntaxe MySQL est la suivante

REVOKE <droit1>, <droit2>, ...
ON [TABLE <nom table>],[*]
FROM <user1>, <user2> ...;

Pour enlever les droits de lecture de la table UTILISATEURS à Mr SMITH par exemple :
REVOKE SELECT ON UTILISATEURS FROM ASMITH;
Pour enlever tous les droits à Mr Smith :
REVOKE ALL PRIVILEGES ON UTILISATEURS FROM ASMITH;
De la même façon que le GRANT, si on ajoute le mot PUBLIC, on supprime les droits sur cette table pour tous les utilisateurs de la base de données.
REVOKE ALL PRIVILEGES ON UTILISATEURS FROM PUBLIC;

2. Supprimer des droits sur les objets de la base
Comme pour la manipulation d'une table, on peut supprimer des droits que l'on a attribués sur la création de table ou la création d'index par exemple.

- Exemple Oracle

REVOKE CREATE TABLE FROM ASMITH;

- Exemple MySQL

REVOKE CREATE ON * ON 'ASMITH';

CHAPITRE 4 : Optimisation de Performance

I. Indexation

1) Présentation

L'indexation est une technique qui améliore les performances des requêtes de base de données en créant et en gérant des structures de données qui permettent un accès plus rapide aux données. Les bases de données SQL utilise un modèle de données relationnelle, où les données sont stockées dans des tables avec des lignes et des colonnes. Pour améliorer les performances des requêtes SQL, qui peuvent joindre, filtrer, trier et agréger des données provenant de plusieurs tables à l'aide de divers opérateurs et fonctions, différents types d'index sont utilisés. Un index est une structure de données qui permet d'accélérer les recherches dans une table en associant à une clé d'index (la liste des attributs indexés) l'emplacement physique de l'enregistrement sur le disque.
Les accès effectués sur un index peuvent donc se faire sur des structures optimisées pour la recherche (liste triée, B-tree...) au lieu de se faire par parcours séquentiel et intégral des enregistrements.
Généralement un index pourra être utilisé dans les requêtes utilisant les clauses WHERE, GROUP BY ou ORDER BY. Lorsqu'une base de données possède un grand nombre d'enregistrements (exemple: plusieurs milliers ou plusieurs millions de lignes) un index permet de gagner un temps précieux pour la lecture de données.
Les index doivent être utilisés sur les tables qui sont fréquemment soumises à des recherches. Ils sont d'autant plus pertinents que les requêtes sélectionnent un petit nombre d'enregistrements (moins de 25% par exemple).
Les index doivent être utilisés sur les attributs :

- souvent mobilisés dans une restriction (donc une jointure),
- très discriminés (c'est à dire pour lesquels peu d'enregistrements ont les mêmes valeurs),
- rarement modifiés.

les index présentent certains inconvénients comme :

- Les index diminuent les performances en mise à jour (puisqu'il faut mettre à jour les index en même temps que les données).
- Les index ajoutent du volume à la base de données et leur volume peut devenir non négligeable.

2) Création des index

En SQL, la commande CREATE INDEX permet de créer un index.
La syntaxe basique pour créer un index est la suivante :
CREATE INDEX `index_nom` ON `table`;
Il est également possible de créer un index sur une seule colonne en précisant la colonne sur laquelle doit s'appliquer l'index :
CREATE INDEX `index_nom` ON `table` (`colonne1`);
L'exemple ci-dessus va donc insérer l'index intitulé "index_nom" sur la table nommée "table" uniquement sur la colonne "colonne1". Pour insérer un index sur plusieurs colonnes il est possible d'utiliser la syntaxe suivante:
CREATE INDEX `index_nom` ON `table` (`colonne1`, `colonne2`);
L'exemple ci-dessus permet d'insérer un index sur les 2 colonnes : colonne1 et colonne2.

Créer un index unique

Un index unique permet de spécifier qu'une ou plusieurs colonnes doivent contenir des valeurs uniques à chaque enregistrement. Le système de base de données retournera une erreur si une requête tente d'insérer des données qui feront doublons sur la clé d'unicité. Pour insérer un tel index il suffit d'exécuter une requête SQL respectant la syntaxe suivante :

CREATE UNIQUE INDEX `index_nom` ON `table` (`colonne1`);

3) Les différents types d'index

Les différents types d'index SQL

Il existe plusieurs types d'index SQL, et chacun est optimisé pour des situations spécifiques. Voici un tour d'horizon des plus courants.

a) index unique

Comme son nom l'indique, l'**index unique** garantit que les valeurs dans la colonne indexée sont uniques. Ceci permet d'accélérer les **recherches par des valeurs uniques**, mais ajoute aussi une contrainte d'unicité. Assure que toutes les valeurs dans l'index sont distinctes. Il est créé avec la commande CREATE UNIQUE INDEX.

CREATE UNIQUE INDEX idx_unique_isbn ON livres (isbn);

b) index composites

Les index composites quant à eux sont créés sur plusieurs colonnes. Ils sont particulièrement utiles pour les requêtes filtrantes ou triant en fonction de plusieurs critères. On peut les utiliser pour améliorer les performances des requêtes de jointure. Il est utile pour des recherches complexes impliquant plusieurs critères.

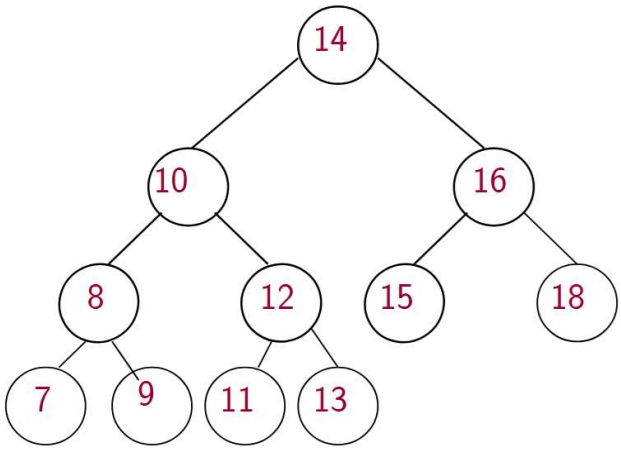
CREATE INDEX idx_auteur_titre ON livres (auteur, titre);

c) Séquentiel (index dense)

Toutes les clés de recherche sont présentes dans l'index.

d) Arbres B

Tout d'abord, l'**Index B-tree ou arbre B** est le plus utilisé. Il organise les données dans un arbre équilibré, permettant des recherches rapides. C'est très efficace pour les requêtes de comparaison comme les opérations d'égalité et de plage. Il facilite les tries comme des recherches par dichotomie.



e) Bitmap

De son côté, l'**Index Bitmap** est utile pour les colonnes avec des valeurs discrètes telles que les catégories. Il utilise des opérations de bits pour accélérer les requêtes complexes, comme

les opérations de filtre avec plusieurs critères. Un index bitmap considère toutes les valeurs possibles pour un attribut. Pour chaque valeur, on stocke un tableau de bits (dit bitmap) avec autant de bits qu'il y a de lignes dans la table. Très utile pour les colonnes qui ne possèdent que quelques valeurs distinctes.

f) Hashage

Les index hash ne peuvent gérer que des comparaisons d'égalité simple. Ils utilisent une fonction(h) qui appliquée à la clé h(clé) donne l'adresse où se trouve l'enregistrement.
La commande suivante est utilisée pour créer un index hash :
CREATE INDEX *nom* ON *table* USING HASH (*column*);

II. Plans d'exécution (EXPLAIN, ANALYZE)

1) Présentation

Le **plan d'exécution** est un schéma pas à pas des opérations d'accès aux données d'un système de gestion de base de données SQL.
Il représente la stratégie utilisée par le moteur de base de données pour exécuter une requête SQL. Le plan décrit les étapes que le SGBD suivra pour récupérer les données demandées, détaillant comment les tables seront accessibles, quels index seront utilisés et l'ordre des opérations. Comprendre le plan est essentiel pour les administrateurs de bases de données et les analystes de données, car il a un impact direct sur les performances et l'efficacité des requêtes.

2) Composants d'un plan d'exécution de requête

Un plan d'exécution de requête typique se compose de plusieurs éléments clés, notamment des opérateurs, des coûts estimés et cardinalité estimations. Les opérateurs représentent les différentes actions que le SGBD va effectuer, telles que les analyses, les jointures et les tris. Les coûts estimés fournissent une approximation des ressources requises pour chaque opération, permettant ainsi une évaluation des performances. Les estimations de cardinalité indiquent le nombre de lignes qui devraient être traités à chaque étape, ce qui est crucial pour optimiser les performances des requêtes.

3) Comment générer un plan d'exécution de requête

La génération d'un plan d'exécution de requête peut généralement être effectuée à l'aide de commandes ou d'options spécifiques dans un SGBD. Par exemple, dans SQL Server, la commande « SET SHOWPLAN_ALL ON » peut être utilisée pour afficher le plan d'exécution sans exécuter réellement la requête. Dans d'autres systèmes comme MySQL, l'instruction « EXPLAIN » a un objectif similaire. En analysant le plan généré, les utilisateurs peuvent identifier les goulots d'étranglement potentiels et les domaines d'optimisation.

4) Interprétation d'un plan d'exécution de requête

L'interprétation d'un plan d'exécution de requête nécessite une connaissance des différents opérateurs et de leurs coûts associés. Chaque opérateur du plan est généralement accompagné d'une représentation graphique ou d'une description textuelle, qui aide les utilisateurs à comprendre le flux de données. En examinant le plan d'exécution, on peut identifier les opérations inefficaces, telles que les analyses de tables complètes ou les jointures inutiles, qui peuvent être optimisées pour améliorer les performances des requêtes.

5) Problèmes courants identifiés dans les plans d'exécution de requêtes

Plusieurs problèmes courants peuvent être identifiés grâce à l'analyse des plans d'exécution des requêtes. Ceux-ci incluent les index manquants, ce qui peut entraîner un ralentissement des performances en raison d'analyses complètes des tables ; des stratégies de jointure sous-optimales, qui peuvent entraîner une consommation excessive de ressources ; et des requêtes mal écrites qui n'exploitent pas efficacement les capacités de la base de données. Reconnaître ces problèmes est la première étape vers l'amélioration des performances globales des requêtes.

6) Optimisation des requêtes à l'aide de plans d'exécution

L'optimisation des requêtes en fonction des informations obtenues à partir des plans d'exécution des requêtes implique plusieurs stratégies. Une approche efficace consiste à créer ou à modifier des index pour prendre en charge les requêtes les plus fréquemment exécutées. De plus, la réécriture des requêtes pour réduire la complexité ou pour utiliser des types de jointure plus efficaces peut améliorer considérablement les performances. L'examen régulier des plans d'exécution dans le cadre de la maintenance de la base de données peut conduire à des améliorations continues de l'efficacité des requêtes.

7) SQL EXPLAIN

Dans le langage SQL, l'instruction EXPLAIN est à utiliser juste avant un SELECT et permet d'afficher le plan d'exécution d'une requête SQL. Cela permet de savoir de quelle manière le Système de Gestion de Base de Données (SGBD) va exécuter la requête et s'il va utiliser des index et lesquels.

a) Syntaxe

La syntaxe ci-dessous représente une requête SQL utilisant la commande EXPLAIN pour MySQL ou PostgreSQL :

```
EXPLAIN SELECT *
FROM `user`
ORDER BY `id` DESC
```

b) Exemple

Pour expliquer concrètement le fonctionnement de l'instruction EXPLAIN nous allons prendre une table des fuseaux horaires en PHP. Cette table peut être créé à partir de la requête SQL suivante :

```
CREATE TABLE IF NOT EXISTS `timezones` (
 `timezone_id` int(10) unsigned NOT NULL AUTO_INCREMENT,
 `timezone_groupe_fr` varchar(50) DEFAULT NULL,
 `timezone_groupe_en` varchar(50) DEFAULT NULL,
 `timezone_detail` varchar(100) DEFAULT NULL,
 PRIMARY KEY (`timezone_id`)
);
```

La requête ci-dessous permet de mieux comprendre la structure et les index de cette table. Imaginons que l'on souhaite compter le nombre de fuseaux horaires par groupe, pour cela il est possible d'utiliser la requête SQL suivante :

```
SELECT timezone_groupe_fr, COUNT(timezone_detail) AS total_timezone
FROM `timezones`
GROUP BY timezone_groupe_fr
ORDER BY timezone_groupe_fr ASC
```

- **Analyse de la requête SQL**

Le retour de cette requête SQL est le suivant :

```
EXPLAIN SELECT timezone_groupe_fr, COUNT( timezone_detail ) AS total_timezone
FROM `timezones`
GROUP BY timezone_groupe_fr
ORDER BY timezone_groupe_fr ASC
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	timezones	ALL	*NULL*	*NULL*	*NULL*	*NULL*	421	Using temporary; Using filesort

Requête SQL avec EXPLAIN sans index

Dans cet exemple on constate les champs suivants :

- **id :** identifiant de SELECT
- **select_type :** type de cause SELECT (exemple : SIMPLE, PRIMARY, UNION, DEPENDENT UNION, SUBQUERY, DEPENDENT SUBSELECT ou DERIVED)
- **table :** table à laquelle la ligne fait référence
- **type :** le type de jointure utilisé (exemple : system, const, eq_ref, ref, ref_or_null, index_merge, unique_subquery, index_subquery, range, index ou ALL)
- **possible_keys :** liste des index que MySQL pourrait utiliser pour accélérer l'exécution de la requête. Dans notre exemple, aucun index n'est disponible pour accélérer l'exécution de la requête SQL
- **key :** cette colonne présente les index que MySQL a décidé d'utiliser pour l'exécution de la requête
- **key_len :** indique la taille de la clé qui sera utilisée. S'il n'y a pas de clé, cette colonne renvois NULL
- **ref :** indique quel colonne (ou constante) sont utilisés avec les lignes de la table
- **rows :** estimation du nombre de ligne que MySQL va devoir analyser examiner pour exécuter la requête
- **Extra :** information additionnelle sur la façon dont MySQL va résoudre la requête. Si cette colonne retourne des résultats, c'est qu'il y a potentiellement des index à utiliser pour optimiser les performances de la requête SQL. Le message "using temporary" permet de savoir que MySQL va devoir créer une table temporaire pour exécuter la requête. Le message "using filesort" indique quant à lui que MySQL va devoir faire un autre passage pour retourner les lignes dans le bon ordre
- **Ajout d'un index**

Il est possible d'ajouter un index sur la colonne "timezone_groupe_fr" à la table.

```
ALTER TABLE `timezones` ADD INDEX ( `timezone_groupe_fr` );
```

L'ajout de cet index va changer la façon dont MySQL peut exécuter une requête SQL. En effectuant la même requête que tout à l'heure, les résultats seront différent.

```
EXPLAIN SELECT timezone_groupe_fr, COUNT( timezone_detail ) AS total_timezone
FROM `timezones`
GROUP BY timezone_groupe_fr
ORDER BY timezone_groupe_fr ASC
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	timezones	index	*NULL*	index_timezone_groupe_fr	153	*NULL*	471	

Dans ce résultat il est possible de constater que MySQL va utiliser un l'index "index_timezone_groupe_fr" et qu'il n'y a plus aucune information complémentaire d'indiquée dans la colonne "Extra".

8) EXPLAIN ANALYZE

EXPLAIN ANALYZE est une commande PostgreSQL qui fournit un plan d'exécution détaillé pour une requête SQL. Ce plan montre comment PostgreSQL prévoit d'exécuter la requête, y compris les opérations effectuées, les index utilisés et les coûts estimés. En utilisant EXPLAIN ANALYZE, vous pouvez identifier les parties de la requête qui prennent le plus de temps et ajuster votre SQL ou vos index en conséquence.
Voici un exemple complexe de ce que peut donner EXPLAIN dans PostgreSQL :

```
Sort  (cost=235.34..236.84 rows=600 width=32) (actual time=8.239..8.242 rows=12 loops=1)
  Sort Key: (avg(e.salary)) DESC
  Sort Method: quicksort  Memory: 25kB
  ->  HashAggregate  (cost=198.50..208.50 rows=600 width=32) (actual time=8.155..8.167
rows=12 loops=1)
        Group Key: d.department_name
        Filter: (count(e.employee_id) > 5)
        ->  Hash Join  (cost=34.00..175.50 rows=1840 width=16) (actual time=0.456..7.123
rows=1750 loops=1)
              Hash Cond: (e.department_id = d.department_id)
              ->  Seq Scan on employees e  (cost=0.00..121.40 rows=1840 width=16) (actual
time=0.012..4.567 rows=1750 loops=1)
                    Filter: (hire_date > '2020-01-01'::date)
              ->  Hash  (cost=22.00..22.00 rows=960 width=8) (actual time=0.235..0.235
rows=950 loops=1)
                    Buckets: 1024  Memory Usage: 45kB
                    ->  Seq Scan on departments d  (cost=0.00..22.00 rows=960 width=8) (actual
time=0.008..0.156 rows=950 loops=1)
Planning Time: 0.345 ms
Execution Time: 8.456 ms
```

L'utilisation de ANALYSE va simplifier le résultat et faciliter son interprétation

Exemple d'utilisation

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM employees WHERE department_id =
1;
Résultat :
Seq Scan on employees  (cost=0.00..18.10 rows=2 width=37) (actual time=0.012..0.014
rows=2 loops=1)
  Buffers: shared hit=4
  Filter: (department_id = 1)
  Rows Removed by Filter: 2
Planning Time: 0.080 ms
Execution Time: 0.032 ms
```

III. Optimisation des requêtes SQL

L'optimisation de requête est l'un des facteurs qui affecte les performances des applications. Les bases de données sont des systèmes très complexes. Afin d'en tirer toutes les performances possibles, l'optimisation doit porter sur un très grand nombre de composants différents : le serveur qui héberge la base de données, et de manière générale tout l'environnement matériel, les processus faisant fonctionner la base de données, les fichiers et disques servant à son stockage, le partitionnement, mais aussi, et surtout, la base de données elle-même. Quelques conseils d'optimisation :

a) Utilisez une indexation correcte

- Indexer les colonnes fréquemment interrogées,
- Éviter d'utiliser des index inutiles,
- Choisir le bon type d'indice.

*b) Évitez le SELECT **

En règle générale, on doit sélectionner que les colonnes spécifiques dont on a besoin. La réduction des données inutiles permet non seulement de conserver un code propre et facile à comprendre, mais aussi d'optimiser les performances.

c) Utiliser efficacement les jointures

d) Analyser les plans d'exécution des requêtes

e) Optimisez les clauses WHERE

- Ajoutez des conditions de filtrage appropriées dès le début ;
- Évitez d'utiliser des fonctions sur les colonnes dans la clause WHERE ;
- Utilisez les opérateurs appropriés.

f) Optimiser les sous-requêtes

g) Surveiller et optimiser les statistiques des bases de données

h) Utiliser des procédures stockées

i) Évitez les classements et les regroupements inutiles

j) Décomposer les requêtes complexes

IV. Partitionnement et organisation physique

A. Partitionnement

Dans une base de données, une **partition** est une division logique d'une table stockée en plusieurs parties indépendantes. Il peut être horizontal (les lignes d'une même table sont divisées en plusieurs partitions) ou vertical (les colonnes d'une même table sont divisées en plusieurs lots). Le partitionnement de tables est généralement effectué pour améliorer la gestion, la performance ou la disponibilité. Chaque partition peut être placée sur des serveurs ou des disques différents. Cela permet également d'obtenir une capacité de base de données supérieure à la taille maximum des disques durs ou d'effectuer des requêtes en parallèle sur plusieurs partitions.

1) Partitionnement vertical

Le partitionnement vertical d'une table divise les données en blocs de colonnes.

2) Partitionnement horizontal

Le partitionnement horizontal d'une table divise les données en blocs de lignes, généralement en fonction de la clef primaire.
Les systèmes de gestion de base de données proposent le partitionnement d'une table selon différents critères. Cela nécessite une *clef de partitionnement* (en général une colonne de la table) et divisent la table suivant la valeur de la clef.
Les critères principaux sont les suivants :

a) Partitionnement par intervalles

Les partitions sont déterminées par des domaines, chaque domaine étant constitué par une plage de valeurs continues. C'est généralement le cas du partitionnement par dates, dont les intervalles peuvent être mensuels, trimestriels, annuels...

b) Partitionnement par liste

Le domaine du partitionnement est une liste de valeurs pour laquelle chaque partition ne contient exclusivement que la seule valeur de la partition. Considérons par exemple le cas d'une table que l'on désire partitionner par devise. La colonne de partitionnement Monnaie permettra de ventiler les lignes de la table suivant différentes devises, par exemple Euro, Dollar US, Livre Sterling, Yen ou Franc CFA ce qui facilitera les calculs de conversion et de change.

c) Partitionnement par hachage

Les partitions sont déterminées par un calcul pseudo aléatoire effectué sur une valeur arbitraire, par exemple un e-mail. On applique alors une fonction de hachage, généralement basique (pour que le calcul soit le plus rapide possible) et l'on calcule le modulo du résultat par rapport au nombre de partitions que l'on souhaite créer. Par exemple, si l'on désire utiliser 4 partitions, on appliquera le modulo 3 au résultat du hachage après avoir rendu le nombre positif (valeur absolue). Un tel calcul s'effectue comme suit :
MOD(ABS(MD5(email)), 4)
Une telle formule de calcul retournera une valeur entre 0 et 3.
Ce type de partitionnement est souvent utiliser pour équilibrer la charge des partitions, dans le sens ou un partitionnement avec un critère sémantique particulier entraine, la plupart du temps, un déséquilibrage des partitions. Dans notre exemple de partitionnement par devise, il y a fort à parier que les partitions "Livre Sterling" et "Yen" devraient avoir une cardinalité bien moins élevée que "Franc CFA" ou "Euro".

d) Partitionnement composite

Le partitionnement composite permet de combiner plusieurs de ces schémas de partitionnement, par exemple en appliquant d'abord un partitionnement par intervalles puis un sous-partitionnement par hachage.

3) Outils de partitionnement des données

Le partitionnement des données peut être mis en œuvre dans les bases de données SQL.

a) PostgreSQL

PostgreSQL offre un support intégré pour le partitionnement par plage, par liste et par hachage. Par exemple, la requête suivante crée une partition pour chaque année afin d'accéder efficacement aux bénéfices des ventes et aux ventes totales pour chaque exercice financier.

```
CREATE TABLE sales(
   city_id        int not null,
   sales_year     date not null,
   total_sales    int,
   sales_profits  int
) PARTITION BY RANGE (sales_year);
```

b) MySQL

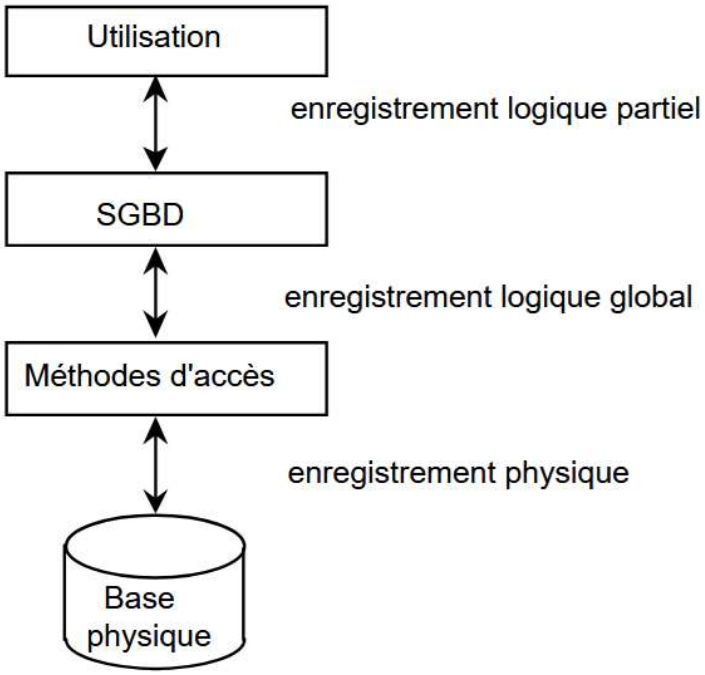
MySQL prend en charge différents types de partitionnement horizontal, tels que le partitionnement par plage, par liste, par hachage et par clé. Jusqu'à la version 8.4), il ne prend pas en charge le partitionnement vertical.
La requête MySQL ci-dessous répartit les données des employés en trois groupes, p1, p2, et p3, en fonction de leur date d'adhésion. Tous les employés qui sont entrés dans l'entreprise avant 2000 font partie de la partition p1. Les salariés qui ont adhéré entre 2000 et 2010 relèvent de la partition p2, et les salariés qui ont adhéré entre 2010 et 2020 relèvent de la partition p3.

```
CREATE TABLE employee(
   firstname VARCHAR(25) NOT NULL,
   username VARCHAR(16) NOT NULL,
   email VARCHAR(35),
   salary INT,
   joined DATE NOT NULL
)
PARTITION BY RANGE( YEAR(joined) ) (
   PARTITION p1 VALUES LESS THAN (2000),
   PARTITION p2 VALUES LESS THAN (2010),
   PARTITION p3 VALUES LESS THAN (2020)
);
```

B. Organisation physique

1) Notion de chemins d'accès

Un SGBD est installé sur un système d'exploitation. Il utilise donc les organisations physiques disponibles à ce niveau (fichiers séquentiels, fichiers relatifs, etc.). Les possibilités d'organisations plus sophistiquées sont spécifiques à chaque SGBD.



Accès aux données dans une base de données

Plusieurs possibilités de chemins d'accès à un enregistrement permettent de minimiser les temps de réponse.
Il existe toujours au moins un chemin d'accès pour un enregistrement : par la clé primaire, avec la méthode d'accès définie sur le fichier correspondant. Il est possible de définir d'autres chemins d'accès à cet enregistrement, à partir des valeurs des autres champs, en vue de faciliter la résolution de certaines requêtes. Les chemins d'accès sont définis a priori lors de la

conception de la base. Il est possible de les remanier ultérieurement dans certains cas seulement. Les critères à considérer pour le choix d'un chemin d'accès :
• le temps de réponse à une recherche élémentaire (en nombre d'Entrées/Sorties) ;
• les facilités de modification, de suppression et d'insertion ;
• le stockage supplémentaire ;
• les facilités de réorganisation ;
• les facilités de reprise après pannes.

2) Méthodes d'accès standards du système d'exploitation

On peut avoir :

- Accès séquentiel ;
- Accès direct :

• accès direct à un enregistrement à partir de son adresse absolue (ex. accès à l'enregistrement N) ;
• accès direct relatif à un enregistrement à partir de son adresse relative (ex. accès à l'enregistrement suivant).

3) Méthodes d'accès non standards

Il s'agit des accès suivants :

- Accès aléatoire (hachage),
- Accès chaîné,
- Accès indexé.